

Scalable and Accurate Test Case Prioritization in Continuous Integration Contexts

Ahmadreza Saboor Yaraghi, Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel Briand, *Fellow, IEEE*

Abstract—Continuous Integration (CI) requires efficient regression testing to ensure software quality without significantly delaying its CI builds. This warrants the need for techniques to reduce regression testing time, such as Test Case Prioritization (TCP) techniques that prioritize the execution of test cases to detect faults as early as possible. Many recent TCP studies employ various Machine Learning (ML) techniques to deal with the dynamic and complex nature of CI. However, most of them use a limited number of features for training ML models and evaluate the models on subjects for which the application of TCP makes little practical sense, due to their small regression testing time and low number of failed builds.

In this work, we first define, at a conceptual level, a data model that captures data sources and their relations in a typical CI environment. Second, based on this data model, we define a comprehensive set of features that covers all features previously used by related studies. Third, we develop methods and tools to collect the defined features for 25 open-source software systems with enough failed builds and whose regression testing takes at least five minutes. Fourth, relying on the collected dataset containing a comprehensive feature set, we answer four research questions concerning data collection time, the effectiveness of ML-based TCP, the impact of the features on effectiveness, the decay of ML-based TCP models over time, and the trade-off between data collection time and the effectiveness of ML-based TCP techniques.

Index Terms—Machine Learning, Software Testing, Test Case Prioritization, Test Case Selection, Continuous Integration



1 INTRODUCTION

Application of Continuous Integration (CI) significantly reduces integration problems, speeds up development time, and shortens release time by allowing software developers to integrate their work more frequently with the mainline codebase rather than with deferred integration. Each integration is automatically built and usually validated by regression testing (a CI cycle), upon completion of which the developers are provided feedback. The execution of regression tests may require significant computational resources and take hours or even days to be completed. The prolonged execution of regression tests can delay the CI cycles and prevent timely feedback to the developers.

Test Case Prioritization (TCP) techniques address the long execution time of regression testing by prioritizing (ranking) the execution of test cases such that faults can be detected as early as possible, i.e., the test cases with high fault detection probability and lower execution times are given higher execution priority. These techniques can be classified into heuristic-based and ML-based techniques. Heuristic-based techniques often make use of code coverage analysis and test execution history. Collecting coverage information is in general challenging and, furthermore, precise coverage information requires dynamic analysis, which is difficult to apply in practice, more particularly so in a CI context, mainly due to computational overhead and

applicability issues [1], [2], [3], [4], [5]. Concerning heuristics based on the execution history, relying solely on such history for TCP may not lead to stable results for complex systems in a CI context [2], [6]. In addition to cost and effectiveness issues, heuristics are often defined statically, and there is no standard procedure to tune them based on new changes. Adapting to new changes is critical for TCP in a CI context, due to the frequently-changing codebases.

ML-based TCP techniques train ML models based on various features collected from different sources, such as execution history, to prioritize test cases. In general, ML techniques enable the training of effective models based on imperfect features. They also can adapt to new changes either through incremental learning [7] or retraining. Thus, many researchers have relied on ML techniques to address TCP in the CI context. According to a recent survey [8], various ML-based TCP techniques have been applied in the CI context. However, existing work has not relied on a comprehensive set of features for training ML models, which is crucial to achieve high effectiveness. They also often used inadequate evaluation subjects that have a low number of failed test cases and a very short regression testing time. Evaluating the costs and benefits of a comprehensive set of features cannot be done on such subjects, as applying TCP techniques is practically inefficient in those cases. Further, none of the existing work reported the cost and time required for collecting features. Thus, despite significant progress, it is still not clear whether or not existing ML-based techniques have reached their full potential for TCP, as they do not take full advantage of all available data sources. Last, several practical questions regarding the application of ML-based techniques remain unanswered, notably what features can be collected and at what cost to

A. Saboor Yaraghi, M. Bagherzadeh are with School of EECS, University of Ottawa, Ottawa, ON K1N 6N5, Canada.

N. Kahani is with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada.

L. Briand holds shared appointments with the SnT Centre for Security, Reliability and Trust, University of Luxembourg, Luxembourg and the school of EECS, University of Ottawa, Ottawa, ON K1N 6N5, Canada.

support ML-based TCP. These questions include: What is the trade-off between using certain features in terms of data collection time and their impact on the effectiveness of ML models for TCP? How often do the ML models need to be retrained to remain useful? Our goal is to provide concrete recommendations regarding these questions.

To address the issues discussed above, we first define a data model characterizing the operational flow of a typical CI environment (e.g., Travis CI). The data model captures the available data sources and their relations at a high level. We then define a set of features based on the data model and a thorough review of the used features in related work. This results in 150 features across nine groups, which can be collected by analyzing five data sources, including build logs, the source code of test cases and its Version Control System (VCS) history, the code of the system under test, and its VCS history.

To investigate the benefits and costs of the 150 features, we conducted a large-scale empirical analysis. To do that, we first defined and developed methods to extract the features based on a popular CI tool, Travis CI, and projects written in the Java programming language. We then designed and conducted an extensive experiment, based on the latest 50 builds of 25 subjects with an adequate number of failed test cases and a regression testing time of at least five minutes, to answer the following questions.

- How does data collection time across feature groups compare, and are they significantly different?
The result shows that data collection time ranges between 0.1 to 11.7 minutes across subjects for each build, the main portion of which is related to features that require static coverage analysis.
- How is the effectiveness of ML-based TCP using the defined comprehensive feature set? How does the use of each feature impact effectiveness?

ML model trained using the full comprehensive set of features can reach promising results for TCP for most study subjects. Test execution history features have the greatest impact on TCP effectiveness.

- How often do the ML-based TCP models need to be retrained? What is the best trade-off between retraining frequency and model effectiveness?
The result shows that retraining ML-based TCP models should be performed no less frequently than every 11 builds, and as frequently as possible to achieve the best TCP effectiveness.
- What are the trade-offs between the data collection time of features and their impact on the effectiveness of ML-based TCP?

Depending on the acceptable degree of data collection overhead and the need for higher effectiveness, we suggest four alternative choices: using the comprehensive set of features for training the ML model, (1) with or (2) without retaining at each build, (3) using only features based on the execution history of test cases for training, and (4) rely on heuristics based on the failure history of test cases.

Overall, this work makes the following contributions:

- Collection and evaluation of a comprehensive feature set for training ML models for TCP. This set includes

all features used by previous studies and addresses all data sources available in the CI context. No previous study is nearly as comprehensive as our work in this respect.

- Answering four important practical questions based on extensive empirical analysis, as discussed above.
- A benchmark of 25 subjects with 21.5k builds and 2.5k failed builds that enables a fair comparison and evaluation of future TCP techniques. We also made our data collection tools available, which can be used to extend and update the subjects¹.

The rest of this paper is organized as follows. Section 2 discusses ML-based TCP in CI context, static coverage analysis, and the classification of source code changes. We review related work in Section 5. Section 3 defines the data and features used in our work and describes how each feature can be practically collected. We then discuss our evaluation approach and results in Section 4, and conclude the paper in Section 6.

2 BACKGROUND

In this section, we first describe how ML-based test case prioritization (TCP) fits into a continuous integration (CI) context. We precisely define the ML problem that our study aims to solve. We then explain the methods that are used to extract dependencies between source code entities and test cases and among source code entities. Further, we present our approach for classifying software systems' changes (commits) into *defect-fix* and *non-defect* classes. The dependency and classification of changes are required for computing coverage-based test case features. More specifically, the dependency data is used for the change and impact analysis features, and the classification data is used for test case coverage features. Details regarding the definition of the features are discussed in Section 3.

2.1 ML-based Test Case Prioritization in CI Context

In a typical CI build, the system source code is built, and then its quality is validated by running a set of regression test cases, whose execution can be costly in terms of time and computation resources. Test Case Prioritization (TCP) techniques address this challenge by prioritizing (ranking) their execution such that test cases with a higher probability of fault detection and lower execution times are given higher execution priority. Depending on the execution budget, top-ranked test cases can be selected for execution or the regression testing stops once a test case fails. Figure 1 depicts a CI build process relying on ML-based TCP. First, similar to a typical CI build, the process begins with building the system. Second, test case features are extracted from several data sources (e.g., system source code) as discussed in Section 3. Third, the extracted features are passed to a pretrained ML ranking model for prediction. This model ranks regression test cases to be executed against the system. The pretrained ML ranking model is trained periodically in an offline environment and therefore does not cause computational overhead to the CI build.

The feature extraction step of ML-based TCP, which occurs before ranking regressions test cases for each build,

1. <https://github.com/Ahmadreza-SY/TCP-CI>

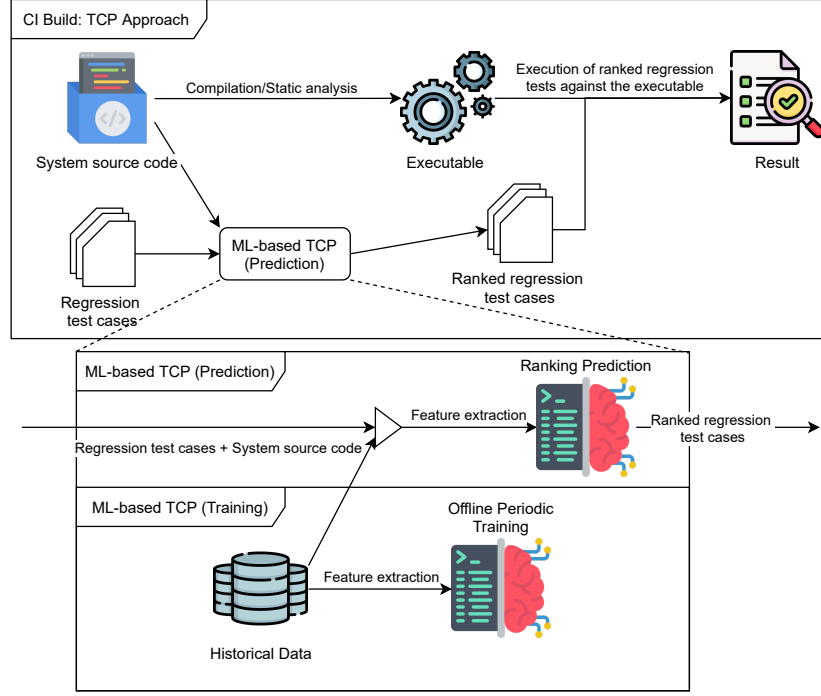


Figure 1: ML-based TCP in CI Context

may significantly affect the overall CI runtime. Though the computation of some test case features only entails a simple analysis (e.g., a database query), other features require code analysis and the computation of metrics, thus possibly delaying the execution of a CI build. This motivates us to investigate the time needed for extracting and computing regression test case features for each CI build and to measure the delay it causes. We investigate this issue in the first research question (RQ1) in Section 4.

Given a test suite T that contains a number of regression test cases, a feature set F that contains features for all test cases, and a ML ranking model M , we define an ML-based TCP as a function that takes M and F as input and produces an ordering of T called T^* that is intended to be as close as possible to the unknown optimal ordering. In this work, similar to existing TCP techniques [9], we assume that in the optimal ordering of test suite T (T^{opt}), test cases are first sorted by their verdict (i.e., fail/pass state after execution) with failed test cases at the beginning. Second, test cases are sorted by their execution time in ascending order. This ensures the detection of faults as early as possible during regression testing. Also, when the execution budget is limited, the execution of top-ranked test cases (Test Case Selection) may be the only option for an efficient use of the budget.

2.2 Dependency and Impact Analysis of Source Code Entities

Code coverage can be calculated in different ways depending on the selected analysis techniques. In this work, for practical reasons related in part to scalability as pointed out by the industry partner supporting this research, we use a mix of lightweight static analysis and association rule mining to calculate the dependency graph between source code files of the System Under Test (SUT) and its test cases,

assuming that test cases are developed using the same language as the SUT. The dependency graph is a directed graph, each node of which refers to a source file. Each edge shows a dependency relation from the source node to the destination, i.e., the source node depends on the destination. The edges are also weighed with coverage scores calculated based on the association rule mining of co-changes of source files corresponding to the source and destination nodes. In the following, we refer to the source code file corresponding to a node by only referring to the latter.

A dependency graph is constructed for a version of the SUT and its test cases in a specific build according to the following steps. First, the algorithm that creates the dependency graph accepts a set of test cases and all changes (commits) of the system up to the target version. It then iterates over all source files and creates a node for each one of them. Edges from a node to other nodes are added if the former either calls a function from the latter or imports them. When the graph is completed, then a dependency score is calculated for each edge based on the association rule mining results of co-changes among nodes. The algorithm iterates over all commits of the development history of a given SUT, extracts all pairs of co-changes which are associated with edges of the dependency graph, and finally calculates the support, confidence, and lift for each edge as discussed below.

Assuming that CH is a list of change sets in the project's commit history: $CH = (\{f_1, f_2, f_3\}, \{f_1, f_3\}, \{f_2\}, \{f_1, f_2, f_3, f_4\}, \dots)$ with f_i representing source files, let us define the following helper functions.

$$\begin{aligned} \text{p_cnt}(f_i, f_j) &= |\{(f_i, f_j) \subseteq E : E \in CH \wedge 1 \leq i, j \leq n \wedge i \neq j\}| \\ \text{cnt}(f_i) &= |\{f_i \in E : E \in CH \wedge 1 \leq i \leq n\}| \end{aligned}$$

where n is the number of source files. Function $p_cnt(f_i, f_j)$ computes the number of commits in which f_i and f_j were changed together. Also, function $cnt(f_i)$ computes the number of commits in which f_i was changed. We define support, confidence, and lift as follows.

$$\text{support}(f_i, f_j) = \frac{p_cnt(f_i, f_j)}{|CH|}$$

$$\text{confidence}(f_i, f_j) = \frac{p_cnt(f_i, f_j)}{cnt(f_i)}$$

$$\text{lift}(f_i, f_j) = \frac{p_cnt(f_i, f_j)}{cnt(f_i) * cnt(f_j)}$$

Definition 1. Coverage. Here, we assume that the test cases are developed using the same language as the SUT. Thus, the source files covered by a test case refer to all source files that the test case depends on, i.e., the test case either calls a function in the related source files or imports them.

The coverage score refers to the confidence of an association rule between a test case and a source file that is computed using association rule mining. To be more precise, the coverage score function (cov_score) is defined as:

$$cov_score(f, t) = confidence(f, t)$$

where f is a source file and t is a test case. In other words, $cov_score(f, t)$ estimates the conditional probability of t being changed given that f is changed.

2.3 Classification of Defect-fix Commits

Definition 2. Previously Detected Faults (PDF). Let us define Previously Detected Faults (PDF) of a source file f as the number of faults detected in f according to its change history:

$$PDF(f) = |\{c \in C | f \in chn(c) \wedge chn_cls(c) = defect_fix\}|$$

where C is the set of all commits of a project, $chn(c)$ is a function that returns a set of changed files for commit c , and $chn_cls(c)$ is a commit classifier that classifies commit c into *defect-fix* or *non-defect* classes.

The classification of commits is still an active research area, initiated by Mockus et al. [10], who used three keyword-based rules to automatically classify modification requests' textual descriptions into four maintenance groups. Hindle et al. [11] applied machine learning models using project-dependent features (e.g., authors, modules, and file types) as well as commits' word distributions. Levin et al. [12] also used a keyword-based approach as well as source code changes from commits. In a recent effort, Zafar et al. [13] created a commit classifier that reached the high accuracy of 92.2% for their test dataset. They used BERT [14], a state-of-the-art text classification method.

BERT [14] is a pretrained transformer-based language model which is trained on massive corpora including BooksCorpus [15] with 800M words and English Wikipedia with 2,500M words. BERT achieved state-of-the-art results on a number of natural language processing tasks. The original $BERT_{BASE}$ model has 110M parameters with 12 encoder layers, and it requires GPU resources for effective training and prediction. For this reason, BERT is computationally expensive and compared to other machine learning

Table 1: Accuracy of two classification models for defect-fix commit classification across four datasets. BERT refers to the model proposed by Zafar et al. [13]. XGBoost refers to our classification model.

Dataset	BERT	XGBoost
Zafar et al. [13]	92.2%	89.2%
Levin et al. [12]	78.0%	75.1%
Berger et al. [19]	81.8%	81.6%
Berger et al. [19] (subset [13])	91.8%	90%

models, this is a major hurdle in a CI context, due to the resource and timing constraints of CI builds.

Thus, we aim to use TF-IDF (term frequency-inverse document frequency) and simpler classification techniques (e.g., Random Forests [16] and SVM [17]) to train a commit classifier that requires much less computation time than BERT [14], while retaining high classification accuracy. TF-IDF [18] measures the importance of a word based on its frequency in a corpus and its presence in individual documents. In the following, we discuss the details of training and evaluating a lightweight commit classifier.

Preparing training datasets. We collected three publicly available datasets in the literature, including Zafar et al. [13], Levin et al. [12], and Berger et al. [19], and used their union as the training dataset. Each dataset includes commit messages and their binary labels indicating whether a commit is a defect-fix or not. The training dataset included 3,681 commits, 35.2% of which were defect-fix commits. We applied a number of preprocessing techniques to clean and normalize the text, which included converting letters to lowercase, replacing URLs with a single token, removing non-alphanumeric characters, removing stop words, and stemming. We then used the TF-IDF technique to convert textual descriptions of commits into vectors.

Training and evaluation of classifiers. Based on the vectors of commits' descriptions, we trained three classifiers using Random Forests [16], SVM [17], and XGBoost [20]. We then evaluated the classifiers using k-fold cross validation with $k=5$, and XGBoost achieved the best accuracy (83.5% of correct predictions). Thus, we selected XGBoost to train our commit classifier in this work.

Comparison with the state-of-the-art. To compare our XGBoost classifier with the one based on BERT (i.e., Zafar et al. [13]), we trained four classifiers based on the four datasets that Zafar et al. [13] experimented with. In Table 1, the BERT column shows BERT's accuracy that was published by Zafar et al. [13], and the XGBoost column shows the average accuracy based on k-fold cross-validation for our XGBoost classifier. As shown in Table 1, BERT outperforms the XGBoost-based classifier by only a few percentage points. As a result, we can clearly benefit, most particularly in a CI context, from the much lower training cost of XGBoost without significantly sacrificing accuracy.

3 DATA MODEL

In this section, we first propose a data model based on the regression testing of a CI build. We then describe a feature model aimed at addressing the TCP problem with machine learning and show how our data model relates to

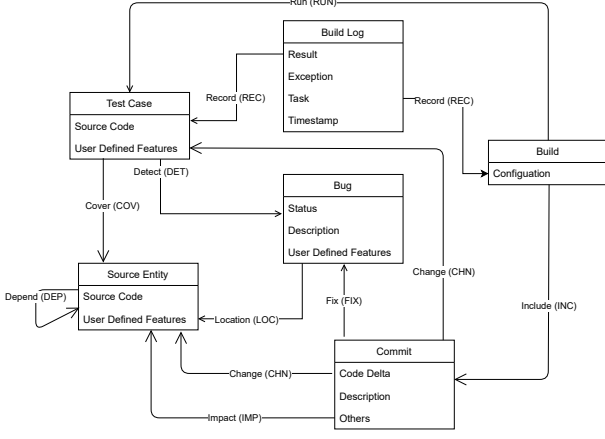


Figure 2: High-level Data Model of a CI Build

the features.

3.1 High-level class diagram

Figure 2 shows a high-level class diagram of entities and properties that are relevant to the regression testing of a CI build. For each CI build, a set of source code files of the SUT are changed (*INC*). To verify whether or not existing functionalities are impacted by changes, a set of regression test cases are executed (*RUN*), the results of which are captured (*REC*) in the build log. When a test case detects a fault (*DET*), the relevant data is often captured using a fault tracking tool such as Bugzilla. A fault is fixed via changes to the source code (*CHN*) of the SUT and recorded in a code versioning system such as *Git*. A change of source code entity may impact other dependent entities (*IMP*). Moreover, each test case causes one or more source entities (files or methods) of the SUT to execute (*COV*).

The data represented by our data model is not necessarily available in a structured form and can be available as source code, text files, or even binary files. In the following, we define each of the entities and discuss how their corresponding data can be collected.

Definition 3. Build. This data source refers to the CI builds that are defined by scripts and configurations. The scripts are written using a Domain Specific Language (DSL) provided by build tools (e.g., Maven). The scripts specify the details (recipe) of the build, often in the form of rules. A build configuration is a collection of settings (e.g., compiler version) that guides how to run the build scripts. For a CI build to be completed (a build task), the build script needs to be executed based on a specific configuration. The build scripts and configurations contain metadata required to understand the build logs. Also, build scripts can be analyzed to extract dependencies between source files.

Definition 4. Build Log. This data source refers to the logs of a CI build. Depending on the underlying build technology, logs may contain different details. Despite such differences, existing build tools typically generate logs that contain the build id, timing information, the result of the build, the output of executed test cases showing whether they passed, and the build configuration (e.g., platform and compiler version). The use of this data requires an analysis of log

files that accounts for the specifics of each build tool.

Definition 5. Test case. This data source refers to the source code of test cases and sometimes their descriptions and user-defined properties. Such data can be collected via the analysis of the source code of test cases or their descriptions. Source code analysis typically requires static analysis tools (e.g., Understand [21]). Also, natural language processing (NLP) techniques can be applied to extract useful data from source code or test case descriptions.

Definition 6. Source code. This data source refers to the source code of the SUT, which can be accessed and analyzed at three levels of granularity: source file, class, method. Similar to the source of test cases, static analysis is required.

Definition 7. Commits. This data source refers to all changes to the SUT source code and test cases. A commit captures a change that is applied to a set of source code files. Existing code versioning tools (e.g., Git) provide APIs to access and analyze information about commits.

Definition 8. Fault. This data source refers to the information about detected faults in the SUT. Existing tools, such as Bugzilla, enable end-to-end tracking of faults that captures when and why a fault is introduced and how it is fixed. In a regression testing context, it is essential to know (1) if a test case reveals a fault during regression testing (regression fault), and (2) how a regression fault is fixed (i.e., commits). While fault tracking tools are widely used, the quality of faults' data is determined by the process followed by development teams to record all relevant details.

3.2 Feature model

An ML-based TCP model takes feature vectors of test cases as input and ranks these test cases. Thus, for any feature to be used for training, it must be a property of test cases. This often requires aggregating and recasting data collected at a different level of granularity. For example, coverage data shows which source files are covered by which test cases, but such data cannot be directly used to define test case features and must be aggregated and recast as a test case property.

We define a comprehensive set of test case features that are grouped into three main groups and nine sub-groups, as shown in the class diagram in Figure 3. In the following, we discuss why each group is considered to be a potentially useful set of features for TCP and how features are calculated based on our data model. Although most of the features can be calculated at the three different granularity levels (method, class, and file), we only address the file level here because most open-source data sources (e.g., *RTPTorrent* [22] and *TravisTorrent* [23]) reported test case execution history for test files and classes rather than methods. Coverage features are also collected here at the file level as, for practical reasons invoked by our industry partner, we want to rely on scalable, light-weight static analysis. File-level analysis can however overestimate test case coverage features as each test case file may contain multiple dependent or independent test case methods. A similar analysis and process can be applied for the class and method levels. Also, we use the naming convention *F_name* for features, where *name* refers to a meaningful name

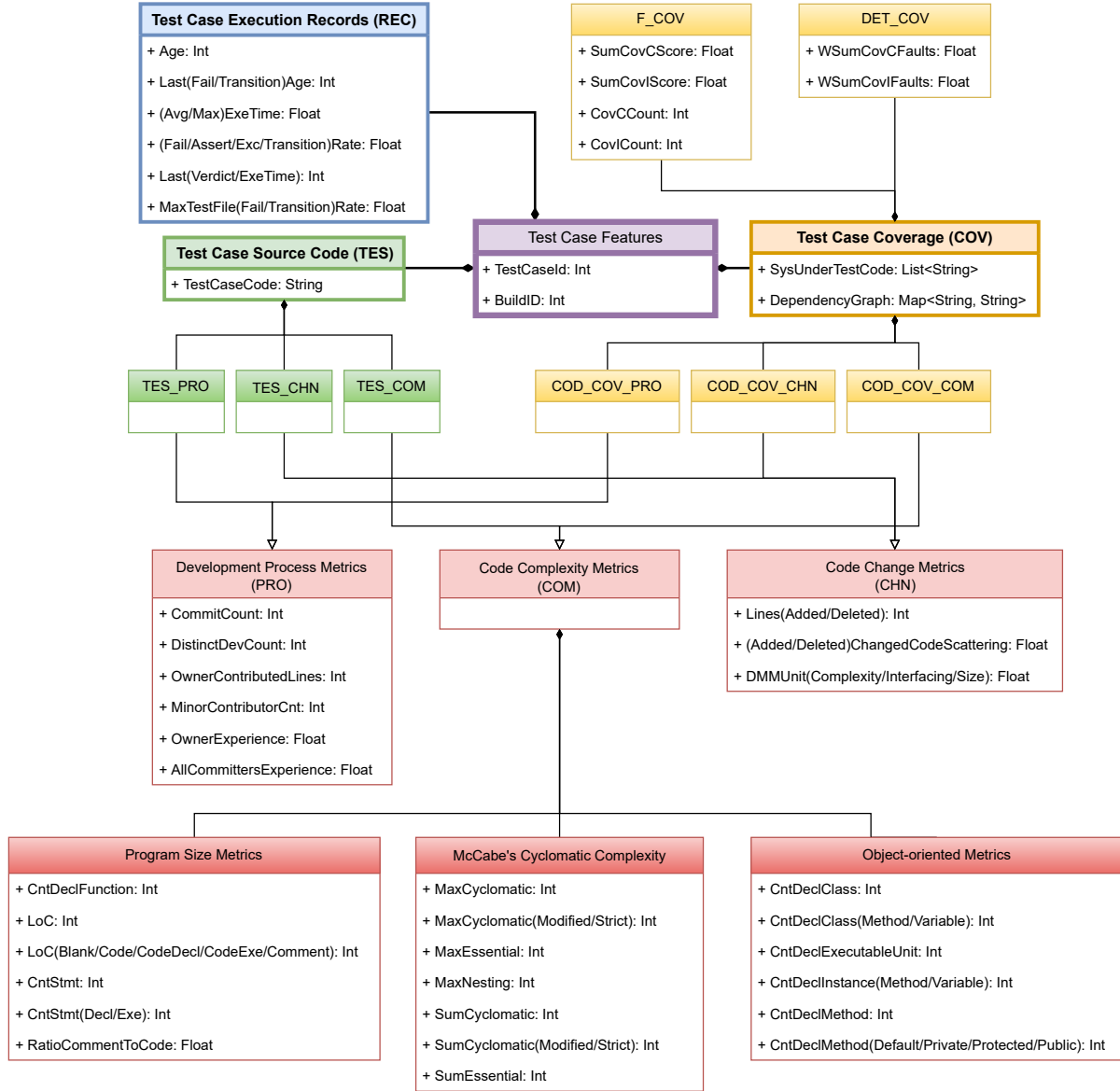


Figure 3: A comprehensive class diagram depicting our proposed feature group hierarchy (three high-level groups with nine low-level groups) and the relation between the feature groups and the metrics used for computing them.

that is selected based on the source code metrics and the aggregation function (if any) used to calculate the feature.

Source code metrics that are grouped into three groups (*complexity*, *process*, and *change*) are presented in Figure 3 and described in the Appendix². *Complexity* metrics measure the complexity of source code as calculated by static analysis. Also, both *process* and *change* metrics concern how the source code has evolved. However, the former is calculated based on the entire change history of the source code, while the latter is only calculated based on the changes of the latest build.

Data collection for each metric can be done in two steps: preprocessing and measurement. During preprocessing, all required data are computed and loaded in memory in a certain format (e.g., Abstract Syntax Tree (AST)) that is adequate for the measurement step, during which the metrics

are calculated. For instance, to collect complexity metrics for a source file, the file is parsed during preprocessing and represented as an AST, based on which the metrics are calculated. Note that metrics in each group share the same preprocessing step and, as we discuss in Section 4, the cost of the measurement step is significantly less than that of preprocessing. This implies that, in practice, the cost of data collection for a metric in a group is close to that of its entire set of metrics.

To facilitate the definition of features below, let us define functions *chn* and *imp* which take a build *b* as input and return two disjoint sets of strictly changed and impacted source code files, respectively. The latter correspond to files that may be affected by changes in other files though they were not changed themselves. Results of functions *chn* and *imp* correspond to relations CHN and IMP in the data model (Figure 2).

2. The Appendix is available as a separate supplementary document.

3.2.1 Test Case Source Code Features (TES)

These features are calculated based on the complexity, process, and change metrics (Figure 3) of the source code files of test cases. The source code features of a test case simply correspond to the metrics of the test case's source file. More details on the definition of metrics can be found in the Appendix. Since this feature group is defined based on source code metrics, we categorize the features in this group into three subgroups, namely TES_COM, TES_PRO, and TES_CHN that correspond to *complexity*, *process*, and *change* metrics, respectively.

The main motivation for using TES features is based on the hypothesis that more complex test cases tend to be associated with longer execution times and a higher probability of detecting faults. Indeed, such test cases are more likely to cover more of the SUT source code and thus have a higher probability of fault detection. Such data are collected through the static analysis of the source code of test cases, which is supported by several tools such as Understand [21], and corresponds to the *Test Case* entity in the data model (Figure 2).

3.2.2 Test Case Execution Record Features (REC)

We define test case features which are calculated based on previous execution time records and verdicts (i.e., failed or passed) of the test cases. These features correspond to relation REC between *Test case* and *Build Log* entities in the data model (Figure 2).

- **F_Age:** This feature captures the number of builds from the first execution of the test case (its introduction). Assuming test case t was first executed on build i , the age of the test case at build n will be equal to $n - i$. We adopt this feature from previous work [24] which reports that newer test cases fail more often since they exercise new and possibly changed source code.
- **F_LastFailAge:** This feature refers to the number of builds from the last failure of the test case [25]. Assuming that the latest failure of test case t occurred on build i , $F_LastFailAge$ of t for build n is equal to $n - i - 1$. $F_LastFailAge$ of a test case that has never failed is set to -1 rather than 0 , the latter being used for a test case that has failed in the previous build.
- **F_LastTransitionAge** [25]: This feature refers to the number of builds since the last change (transition) in the test case's verdict, from failed to passed or vice versa. It is computed in the same way as $F_LastFailAge$ but based on the last transition.
- **F_AvgExeTime:** This feature refers to the average of the previous execution time records of the test case.
- **F_MaxExeTime:** This feature refers to the maximum value of execution time records of the test case.
- **F_FailRate:** This feature is defined as $\frac{f}{n}$, where f is the number of failed executions of the test case and n is the total number of executions of the test case. In this work, we use this rate rather than the failure count of a test case since the latter can be misleading as it is very much dependent on the number of times a test case was executed.
- **F_AssertRate:** An assertion failure of a test case refers to a failure due to an unexpected output of the SUT.

This feature refers to the rate of assertion failures of the test case.

- **F_ExcRate:** An exception failure is caused by an exception that is not handled correctly in the source code of the test case. This feature refers to the rate of exception failure of a test case. An exception failure may indicate a fault in the source code of t rather than that of the SUT. Thus, we distinguish assertion failures from exception failures.
- **F_TransitionRate** [25]: This feature refers to the rate of transitions of the test case verdicts.
- **F_LastVerdict:** This feature captures the verdict of the last execution of a test case.
- **F_LastExeTime:** This feature refers to the last execution time of a test case.
- **F_MaxTestFileFailRate** [25]: Assuming that $TFF(t, f, k)$ refers to the number of builds before build k , in which the file f is changed and test case t has failed, and that $TF(t, k)$ refers to the total number of builds before build k in which test case t has failed, $F_MaxTestFileFailRate$ for test case t in build k with a set of changed files F is calculated as:

$$\frac{\text{Maximum of } TFF(t, f, k), f \in F}{TF(t, k)}$$

If t has never failed, the feature is set to -1 rather than 0 , the latter being used when no failure of t co-occurred with changes of the files that are changed in build k . The use of this feature is motivated by the fact that, if the previous changes in a file are associated with previous failures of a test case, then the future changes of the file are likely to be associated with failures of the test case.

- **F_MaxTestFileTransitionRate** [25]: This feature is exactly computed the same way as $F_MaxTestFileFailRate$ except that it accounts for test case transitions instead of failures.

Due to the frequent execution of regression test cases in CI contexts, the volume of execution history is continuously and quickly growing. Thus, REC features such as $F_AvgExeTime$, $F_MaxExeTime$, $F_FailRate$, $F_AssertRate$, $F_ExcRate$, and $F_TransitionRate$ are typically calculated based on the latest n test case executions, which are extracted by processing the logs of the latest n builds. Since the main goal of our work is to use a comprehensive set of features, we calculate two values for these six features in our final feature set: *Recent* and *Total*. The *Recent* value is computed based on the latest six builds, similar to previous studies [9], [2], while the *Total* value is calculated based on all available builds. Previous work [2] reports that using long text execution history may lead to a reduction in performance.

The primary motivation for using REC features related to fault detection (i.e. $F_FailRate$, $F_AssertRate$, $F_ExcRate$, $F_TransitionRate$, and $F_LastVerdict$) is that test cases that detected more faults in the past are more likely to detect faults again in the future, as they tend to exercise complex and frequently changed features. Thus, we conjecture that past failed test cases should be executed again in new builds. Additionally, the hypothesis behind using execution time history is that test cases that take more time to run are

more likely to execute more complex and compute-intensive code, as well as larger parts of the code. Therefore, long-running test cases are more likely to detect faults in the SUT. Also, since the execution time of test cases is used as the second criteria for prioritizing test cases (when two test cases have the same verdict, the one with the lower execution time is ranked first), it is an important feature for training ML-based TCP techniques.

3.2.3 Test Case File Coverage Features (F_COV)

These features are calculated based on source files covered (exercised) by the test case and correspond to relation COV in the data model (Figure 2). These features capture the ability of test cases to cover source files that are changed (relation CHN) or impacted (relation IMP) in the build.

Let us define function *cov* that takes a source file and a test case as inputs and returns whether the test case covers (exercises) the source file. Also, let us define function *cov_score* that takes the same inputs and returns the normalized coverage score if the test case covers the source file, and zero otherwise. Since we are eventually going to use coverage scores for weighted summations in builds, we need to use normalized scores. Given a set of coverage scores $S = \{s_1, \dots, s_n\}$ (e.g., coverage scores of all changed files covered by a test case in a build), we normalize them by dividing each score by the sum of all coverage scores. Hence, the normalized coverage score set, SN , is defined as follows.

$$SN = \left\{ \frac{s_1}{\sum_{i=1}^n s_i}, \dots, \frac{s_n}{\sum_{i=1}^n s_i} \right\}$$

The four features in this group refer to the number of covered files and coverage score as defined in the following.

- **F_SumCovCScore** of a test case t in build b refers to the sum of coverage scores of t w.r.t the changed source files in build b : $\sum_{f \in \text{chn}(b)} \text{cov_score}(f, t)$.
- **F_SumCovIScore** of a test case t in build b refers to the sum of coverage scores of t w.r.t the impacted source files in build b : $\sum_{f \in \text{imp}(b)} \text{cov_score}(f, t)$.
- **F_CovCCount** of a test case t in build b refers to the number of covered source files by t that are changed in build b : $|\{f : f \in \text{chn}(b) \wedge \text{cov}(f, t)\}|$.
- **F_CovICount** of a test case t in build b refers to the number of covered source files by t that are impacted in build b : $|\{f : f \in \text{imp}(b) \wedge \text{cov}(f, t)\}|$.

Impact analysis is relatively time-consuming and therefore separation of the features based on impacted and changed source files allows us to investigate whether or not including impacted files brings significant benefits. Using such coverage scores as features is motivated by the hypothesis that a test case with higher coverage is more likely to exercise faults, and is, therefore, more likely to detect them.

3.2.4 Features of Covered Source Code by Test cases (COD_COV)

This group includes the source code features of the changed and impacted source files (*Source Entity* in the data model in Figure 2) covered by the test cases. These features are calculated based on the complexity, process, and change metrics in Figure 3. Similar to the TES feature group, we categorize features of this group into three subgroups which are

COD_COV_COM, COD_COV_PRO, and COD_COV_CHN, which correspond to *complexity*, *process*, and *change* metrics, respectively.

Since most test cases cover more than one source code entity, we use the normalized weighted sum of metrics based on the coverage score of the test case w.r.t to source files. Also, similar to F_COV features, we define separate features for changed and impacted source files. For example, the weighted summation of the *LoC* features of covered files of test case t in build b is calculated as follows, assuming that function $\text{loc}(f)$ computes *LoC* for source file f ,

$$F_WSumCLOC = \sum_{f \in \text{chn}(b)} \text{cov_score}(f, t) * \text{loc}(f)$$

$$F_WSumILOc = \sum_{f \in \text{imp}(b)} \text{cov_score}(f, t) * \text{loc}(f)$$

where $F_WSumCLOC$ and $F_WSumILOc$ refer to the features based on changed and impacted files, respectively. As discussed above, other metrics can replace $\text{loc}(f)$ in the above equations.

The main motivation for using COD_COV features based on complexity metrics is due to the hypothesis that the cumulative complexity of covered source files by a test case is an indicator of the execution time and fault revealing power of the test case. Also, change metrics, specifically delta maintainability metrics (DMM) [26], assess the maintainability implications of changes by estimating the risk level of each change. Thus, features defined based on maintainability metrics are good indicators of risky changes, i.e., changes that are likely to be faulty. The same argument applies to features based on process metrics since they indicate the risk entailed by changes by relying on metrics that concern the development process. For example, a change by a new developer has a higher probability of being faulty than that of an experienced developer. Thus, the execution of test cases that cover source files including higher risk changes in the current build has a higher fault detection probability in the context of regression testing.

3.2.5 Test Case Coverage Fault Detection Features (DET_COV)

These features are defined based on the Previously Detected Faults (PDF, Definition 2) of source files that is captured by relations DET and LOC in the data model (Figure 2). We define the following features in this feature group:

- **F_WSumCovCFaults** refers to the weighted sum (weighted by coverage scores) of PDFs of the changed source files covered by test case t in build b .

$$\sum_{f \in \text{chn}(b)} \text{PDF}(f) * \text{cov_score}(f, t)$$

- **F_WSumCovIFaults** refers to the weighted sum (weighted by coverage scores) of PDFs of the impacted source files covered by test case t in build b .

$$\sum_{f \in \text{imp}(b)} \text{PDF}(f) * \text{cov_score}(f, t)$$

The main motivation for using DET_COV features is based on the hypothesis that a source file with a higher PDF is more likely to contain faults in the future. Similarly, a test

case that covers files with higher PDF is more likely to detect faults.

4 VALIDATION

This section reports on the experiments we conducted to assess the impact of features on the effectiveness and cost of TCP techniques. We first discuss and motivate four research questions. We then describe the subjects of the study and the evaluation metrics. Finally, we explain our experimental process, present the results, and discuss their practical implications.

4.1 Research Questions

4.1.1 RQ1. Data Collection Time

- **RQ1.1** How does data collection time across feature groups compare and are they significantly different?
- **RQ1.2** How does accounting for impacted files affect data collection time for each feature group?
- **RQ1.3** How do subject size (Source Lines of Code), the number of test cases, and the number of builds affect the data collection time?

In a CI context, data collection time is a particularly sensitive issue as time is usually strictly limited for regression testing. A feature can be used to train an ML-based TCP if it can be collected within much less time than that of regression test execution. Thus, we define RQ1 to investigate the data collection time of each feature group according to two modes, based on whether or not impacted files are considered (RQ1.1 and RQ1.2). Also, RQ1.3 investigates how the size of subjects in terms of SLOC (Source Lines of Code), the number of test cases, and the number of builds affect data collection times. In particular, RQ1.3 investigates how data collection time increases as the size of the subject grows and whether scalability issues can be expected for large systems when collecting features. This RQ focuses on feature groups since all features in each group rely on the same preprocessing, which accounts for most of the data collection time.

4.1.2 RQ2. TCP Effectiveness

- **RQ2.1** How effective is test case prioritization when using the full feature set?
- **RQ2.2** How effective is test case prioritization when impacted files are not considered?
- **RQ2.3** How does the use of each feature group contribute to the effectiveness of the ML-based TCP?
- **RQ2.4** Which specific feature or subset of features has the highest impact on the effectiveness of ML-based TCP models?
- **RQ2.5** How does the effectiveness of heuristic-based TCP models compare to ML-based models?

RQ2.1-2.4 are particularly important in the context of CI as the features used for ML-based TCP models must be minimized, especially features that induce high data collection time, without significantly improving TCP effectiveness. Therefore, the goal is to investigate what TCP effectiveness can be achieved with all features (RQ2.1 and RQ2.2) and what feature groups are most important for TCP effectiveness (RQ2.3). Though the analysis in RQ2.3 is based on feature groups, within feature groups, even those with low impacts, some individual features may have a higher

impact on effectiveness than others. This is addressed by RQ2.4. Finally, RQ2.5 compares the results of ML-based TCP with heuristic-based TCP to investigate whether or not using ML brings significant advantages that justify its use.

4.1.3 RQ3. How often do the ML-based TCP models need to be retrained?

ML-based TCP models, specifically in the context of CI, need to be retrained regularly based on newly collected data, to better reflect the current status of the system and its history. However, retraining can be expensive and incur delays, and thus we should investigate how the effectiveness of ML-based TCP models decays over time when features are not updated and the models not retrained. This analysis will provide us with insights on how often feature data needs to be collected and used for retraining ML-based TCP.

4.1.4 RQ4. Trade-off between data collection time and TCP effectiveness for features

RQ1 and RQ2 separately investigate the data collection time and effectiveness of the TCP model based on individual feature groups. For engineers to make informed decisions in specific contexts, regarding which feature groups should be used for ML-based TCP, a trade-off often needs to be made between data collection times and effectiveness. Thus, RQ4 conducts a comprehensive trade-off analysis with the objective of providing concrete guidelines regarding the use of feature groups, that will hopefully lead to acceptable effectiveness within reasonable time for a specific context.

4.2 Subjects

We ran our experiments on 25 subjects, which were selected in a 6-step process, as discussed in the following.

- 1) We started with the latest available database of open-source projects from *GHTorrent* [27] (dump 2021-03-06 with more than 100 million projects). We then selected active (i.e., not forked and deleted) and popular (with at least 50 stars) Java projects from the database that resulted in 22,551 projects. *GHTorrent* provides regularly updated databases of GitHub open-source repositories along with tooling to search in the databases of active (i.e., not forked or deleted) projects in GitHub.
- 2) We selected projects with at least 100 CI runs hosted on Travis CI that resulted in 3,323 projects. We then used *TravisTorrent* [23] to fetch CI build logs and commits of the selected open-source repositories. *TravisTorrent* provides scripts for fetching data from Travis CI and GitHub (for build commits).
- 3) We selected projects that use *Maven* as their test execution tool since it provides build logs that contain the required information regarding test case executions (i.e., verdict, duration, and class name). This further reduced the number of projects to 1,419.
- 4) From the resulting projects in step 3, we selected the union of the top 300 projects with the highest SLOC (Source Lines of Code) and the top 300 projects with the highest number of builds. This resulted in 434 projects.
- 5) We used *TravisTorrent* [23] to extract the build logs and build commits of the projects resulting from step 4. We analyzed the build logs to calculate the regression testing duration and failed builds of projects. We then

Table 2: The statistics of the 25 carefully selected subjects. The statistics include source lines of code (SLOC), number of commits, length of the time period between the first and last build, number of all builds and failed builds with the failure rate, average number of test cases per build, and average execution time of all test cases per build.

S_{ID}	Subject	SLOC	Java SLOC	# Commits	Time period (months)	# Builds	# Failed Builds	Failure Rate (%)	Avg. # TC/Build	Avg. Test Time (min)
S_1	JMRI/JMRI	4.56M	1.05M	69.3k	5	1,481	65	4	4364	25
S_2	apache/airavata	1.46M	731k	10.0k	15	236	83	35	49	6
S_3	SonarSource/sonarqube	899k	398k	31.8k	18	4,286	230	5	1309	6
S_4	apache/sling	695k	388k	47.4k	7	1,403	343	24	189	6
S_5	camunda/camunda-bpm-platform	653k	395k	20.6k	34	822	125	15	569	23
S_6	facebook/buck	586k	384k	26.3k	10	846	130	15	663	26
S_7	apache/shardingsphere	422k	165k	29.6k	7	1,049	123	11	789	17
S_8	b2ihealthcare/snow-owl	373k	212k	13.4k	2	277	21	7	46	10
S_9	Angel-ML/angel	336k	204k	3.0k	23	308	124	40	33	20
S_{10}	apache/logging-log4j2	313k	166k	12.7k	13	441	122	27	544	8
S_{11}	eclipse/jetty.project	282k	199k	25.0k	2	192	89	46	137	6
S_{12}	optimatika/ojAlgo	246k	84k	1.6k	22	254	72	28	136	9
S_{13}	yamcs/Yamcs	229k	123k	5.6k	24	504	61	12	114	6
S_{14}	eclipse/steady	221k	98k	2.0k	13	675	51	7	81	7
S_{15}	Graylog2/graylog2-server	182k	85k	22.3k	53	3,668	124	3	110	20
S_{16}	CompEvol/beast2	159k	83k	3.0k	85	415	115	27	65	6
S_{17}	EMResearch/EvoMaster	158k	25k	4.1k	7	583	109	18	100	12
S_{18}	apache/rocketmq	135k	100k	2.0k	16	536	56	10	182	17
S_{19}	zolyfarkas/spf4j	125k	79k	32.6k	37	587	180	30	116	7
S_{20}	spring-cloud/spring-cloud-dataflow	104k	54k	3.5k	9	408	19	4	115	17
S_{21}	cantaloupe-project/cantaloupe	98k	77k	4.5k	29	450	65	14	148	11
S_{22}	thinkarelius/titan	85k	40k	5.1k	25	384	41	10	45	48
S_{23}	apache/curator	84k	58k	3.1k	21	517	65	12	115	67
S_{24}	jcabi/jcabi-github	61k	32k	2.8k	29	788	6	< 0.01	176	14
S_{25}	eclipse/paho.mqtt.java	61k	34k	1.0k	16	378	77	20	37	15

Table 3: The number of frequent-failing (FF) test cases and subject statistics before (B) and after (A) removing the FF test cases. Subjects with no FF test cases are excluded. The subjects that have less than 50 failed builds after removing FF test cases are shown in bold.

S_{ID}	# FF TCs	# Failed Builds		Failure Rate (%)		Avg. # TC/Build		Avg. Test Time (min)	
		B	A	B	A	B	A	B	A
S_5	8	174	125	21	15	575	569	24	23
S_3	7	299	230	6	5	1315	1309	7	6
S_7	6	151	123	14	11	795	789	17	17
S_1	5	94	65	6	4	4368	4364	26	25
S_{17}	2	143	109	24	18	101	100	13	12
S_{21}	2	70	65	15	14	149	148	12	11
S_{22}	2	60	41	15	10	45	45	48	48
S_{24}	2	76	6	9	< 0.01	174	176	13	14
S_4	1	697	343	49	24	189	189	6	6
S_8	1	58	21	20	7	47	46	22	10
S_{10}	1	231	122	52	27	545	544	8	8
S_{11}	1	150	89	78	46	138	137	6	6
S_{13}	1	72	61	14	12	115	114	7	6
S_{19}	1	277	180	47	30	117	116	7	7
S_{20}	1	88	19	21	4	115	115	17	17
S_{23}	1	103	65	19	12	115	115	68	67

selected projects with at least 5 minutes of average regression testing time and 50 failed builds, which resulted in 18 projects. There is no or little practical value in performing TCP when the regression testing time is less than 5 minutes, as most TCP techniques often require more than a few minutes for the data collection and prioritization of test cases. Also, we require at least 50 failed builds to create sufficiently balanced datasets.

- 6) We selected another 7 projects with at least 5 minutes of average regression testing time and 50 failed builds from the 20 open-source projects provided by RTPTorrent’s [22], a public dataset for the evaluation of TCP techniques. Thus, overall we selected 25 projects as the subjects of our studies that are shown in Table 2.

We investigated the failure frequency of test cases among failed builds. For most of the subjects, some of the test cases failed frequently across failed builds. We then investigated the reasons for the existence of such test cases by going through the build logs of a sample of the subjects and reading the error messages caused by test case failures.

In the investigation sample, which included S_{24} , S_{20} , S_8 , and S_7 , we found test cases that failed in more than 65% of the failed builds due to the same reason. Such reasons included external exceptions which were not related to the SUT, such as errors in HTTP requests to external APIs due to authentication issues, invalid arguments, or unexpected responses. Common failure causes also included Java runtime errors, such as *ClassNotFoundException* or *FileNotFoundException*, which were due to missing classes or missing files in the project. Executing such frequently-failing (FF) test cases, which are also referred to as *known breakages*, has no practical value in regression testing as they tend to fail most of the time for the same reason, independently of changes. Since our focus is regression testing, we removed these test cases by performing outlier tests, with respect to failure frequency, using the three-sigma rule of thumb[28]. As suggested by our analysis, outlier test cases are highly likely to correspond to non-regression failures and therefore tend to blur our empirical results. Note that though such outlier tests may not remove all of FF test cases, we are confident that most of them were identified and excluded from our datasets. Hence, the subject statistics, experiments, and results presented throughout the rest of the paper are based on data in which the FF test cases are removed.

The final 25 subjects were selected by analyzing more than 20,000 popular open-source Java projects. The selection process assures that all subjects have an acceptable number of failed test cases and regression testing time, both of which are critical for the application and evaluation of TCP techniques. Table 2 shows the characteristics of the subjects in terms of line of codes, the number of (failed) builds, failure rate, commits, and the average regression testing time per build. The first column (S_{ID}) of the table shows the identifier of subjects that will be used to refer to them in the rest of this section.

Column *SLOC* of Table 2 shows the total number of code and comment lines of the subjects that were counted based on the latest build. *SLOC* ranges from 61k to 4.56M, with a median of 229k. Compared with the subjects that are used

in most recent studies, our work relies on a high number of subjects (25) whose median SLOC is 229k compared to 37.4k [9] and 132k [25], respectively. Also, the average number of test cases per build across our subjects ranges from 33 to 4368, with a median of 117, which is similar to 18 previous studies reported by [8]. Thus, compared to the previous studies, we use a higher number of subjects whose size in terms of SLOC can be considered to be reasonably larger. A build can fail for several reasons, including compilation errors or a test case failure. However, in a TCP context, we are only interested in the latter, and failed builds are, in our subjects, builds with at least one failed test case. Column # *Failed Builds* of Table 2 shows the number of failed builds of each subject. This number ranges from 6 to 343, with a median of 83, representing a diverse set of subjects allowing us to conduct a large number of experiments to account for randomness and draw statistically valid conclusions. Pan et al. [8] reports that previous studies evaluate their work mainly based on subjects with a low number of failed builds that ranges between 1 and 70 with a median of 9, which results in unbalanced training datasets. Also, it is not meaningful to evaluate TCP techniques based on a subject with a very low number of failed builds as the main goal of TCP is the early detection of faults and most of the evaluation metrics are based on counting detected faults.

The last column of Table 2 shows the average of regression testing time per build across all subjects that ranges from 6 to 67 minutes with a median of 12. Compared to recent studies where 11 out of 23 subjects [25] have regression testing times below 3 minutes, or all subjects have regression testing times below 30 seconds [9], our subjects are significantly better. Recall that it is not meaningful to apply and evaluate TCP techniques in the context of projects whose regression testing times are small.

As discussed earlier, we removed the *FF* test cases from the subjects using the outlier test. Table 3 compares the statistics of subjects, which have at least one *FF* test case, before and after removing their *FF* test cases. Column # *FF TCs* corresponds to the number of *FF* test cases, and the rest of the columns show the statistics before (B) and after (A) removing all *FF* test cases. As visible, the number of failed builds for some subjects significantly drops when the *FF* test cases are removed. More specifically, for four of the subjects ($S_{8,20,22,24}$), the number of failed builds decreases to under 50 (our selection criterion) after removing *FF* test cases. This suggests that *FF* test cases were the main cause of build failures across these subjects. However, the effect of removing *FF* test cases on the average number of test cases per build and the average regression testing time per build is negligible.

4.3 Evaluation Metrics

In this work, we use Cost-cognizant Average Percentage of Faults Detected (APFD_C) [29] as the evaluation metric to measure the effectiveness of prioritization techniques. APFD_C is a cost-aware variant of the well-known and widely used APFD [30] metric. APFD only measures the extent to which a certain ranking reveals faults early and does not take the execution time of test cases into account, which is important, especially in a CI context. Similar to prior work [25], [31], [32], since fault severity information is

not available, we assume all faults have the same severity. Also, since our collected data does not include the mapping of faults and test cases, similar to prior work [25], [31], [32], we assume that each test case failure refers to a distinct fault in the SUT. In practice, obviously, this widely used assumption is not correct. However, we can expect the number of faults detected to be roughly proportional to the number of failures.

APFD_C of a test case ordering T^* is calculated as:

$$\text{APFD}_C = \frac{\sum_{i=1}^m (\sum_{j=TF_i}^n t_j - \frac{1}{2}t_{TF_i})}{\sum_{j=1}^n t_j \times m}$$

where m refers to the total number of faults, n refers to the total number of test cases in T^* , and TF_i refers to the position (starting from 1) of the first failed test case in T^* that detects the i th fault, and t_j refers to the execution time of the j th test.

Both APFD and APFD_C can only be computed for builds that contain failures. Thus, here we only report APFD_C based on failed builds.

4.4 Experiment Design, Results, and Discussion

4.4.1 Data collection Time (RQ1)

Overview Table 4 reports the preprocessing (sub-column P) and measurement (sub-column M) times of feature groups across subjects. For computing each test case feature in a CI build, a number of preprocessing steps are required including static source code and dependency analysis, source code change history collection, and text classification. The preprocessed data is used for one or more feature groups. The measurement time refers to the computation of feature values using the preprocessed data. Most of the measurement times, for all feature groups, are less than a second and therefore negligible. In contrast, preprocessing is expensive, taking for many feature groups almost all the data collection time. Such groups require static source code and dependency analysis as well as textual classification. Therefore, in most production environments where systems tend to be as large or larger than the largest system we consider here (S_1), it might be more practical to do the preprocessing periodically rather than on each build, as we will discuss in RQ3. For this RQ, however, we performed the preprocessing for all builds, to account for the worst-case situation in terms of data collection time.

Also, Table 4 shows the average data collection times of feature groups for all builds, across subjects. The first result of practical importance is that the data collection times of certain feature groups are significantly higher than others: COD_COV_COM, COD_COV_PRO, DET_COV, COD_COV_CHN, and F_COV with total data collection time averages of 84, 69, 68.4, 68.9, and 67.3 seconds per build, respectively, with a maximum value (S_1) above 450 seconds. This can be explained by the fact that these feature groups require static source code and dependency analysis. Conversely, TES_CHN, with an average of 1.3 seconds per build, shows the lowest data collection time, since this feature group is based on analyzing test case source code changes in a build, and does not require any historical data or preprocessing. Further, the source code of regression test cases is not frequently changed.

Table 4: Average preprocessing (P), measurement (M), and total (T) data collection time (in seconds) for all feature groups across subjects. For each column, the maximum value is shown in bold.

S_{ID}	COD_COV_COM			COD_COV_PRO			DET_COV			COD_COV_CHN			F_COV			TES_COM			TES_PRO			REC			TES_CHN		
	P	M	T	P	M	T	P	M	T	P	M	T	P	M	T	P	M	T	P	M	T	P	M	T	P	M	T
S_1	508.9	0.4	509.3	455.8	5.5	461.3	452.8	4.1	456.9	451.4	6.1	457.5	451.4	0.0	451.4	57.5	0.1	57.6	4.4	28.9	33.3	0.4	139.1	139.4	0.0	5.6	5.6
S_2	61.6	0.0	61.6	23.9	0.2	24.2	23.6	0.1	23.7	23.3	2.2	25.5	23.3	0.0	23.3	38.3	0.0	38.3	0.6	0.4	1.0	0.0	0.5	0.5	0.0	0.4	0.4
S_3	262.7	0.1	262.8	244.2	0.8	245.0	244.1	0.5	244.6	243.8	4.5	248.4	243.8	0.0	243.8	18.9	0.0	18.9	0.4	5.1	5.5	0.1	21.9	22.0	0.0	4.2	4.2
S_4	98.6	0.0	98.6	74.3	0.2	74.5	74.1	0.1	74.3	74.1	1.1	75.2	74.1	0.0	74.1	24.6	0.0	24.6	0.2	0.9	1.2	0.0	1.3	1.4	0.0	1.1	1.1
S_5	130.0	0.1	130.1	107.9	0.7	108.6	107.7	0.4	108.1	106.8	1.3	108.1	106.8	0.0	106.8	23.2	0.0	23.2	1.1	3.6	4.7	0.1	6.6	6.7	0.0	1.2	1.2
S_6	69.7	0.1	69.9	49.4	0.7	50.1	49.2	0.3	49.5	48.9	1.6	50.5	48.9	0.0	48.9	20.9	0.0	20.9	0.5	3.3	3.8	0.1	5.6	5.7	0.0	1.5	1.5
S_7	110.3	0.1	110.3	93.8	0.6	94.3	93.2	0.3	93.5	92.5	1.6	94.1	92.5	0.0	92.5	17.7	0.0	17.8	1.2	3.3	4.5	0.1	6.2	6.3	0.0	1.5	1.5
S_8	47.5	0.0	47.5	33.5	0.3	33.8	32.8	0.2	33.0	31.2	0.7	31.9	31.2	0.0	31.2	16.2	0.0	16.2	2.3	0.5	2.8	0.0	0.6	0.6	0.0	0.6	0.6
S_9	29.4	0.0	29.4	21.0	0.1	21.1	20.8	0.1	20.9	19.6	0.3	20.0	19.6	0.0	19.6	9.8	0.0	9.8	1.4	0.2	1.6	0.0	0.3	0.3	0.0	0.3	0.3
S_{10}	41.0	0.0	41.1	30.2	0.2	30.4	30.1	0.1	30.1	29.8	0.5	30.2	29.8	0.0	29.8	11.2	0.0	11.3	0.5	2.5	3.0	0.1	3.7	3.8	0.0	0.4	0.4
S_{11}	65.3	0.0	65.3	52.2	1.1	53.2	52.0	0.8	52.9	51.6	0.6	52.3	51.6	0.0	51.6	13.7	0.0	13.7	0.5	1.7	2.2	0.1	1.4	1.4	0.0	0.6	0.6
S_{12}	22.4	0.0	22.4	16.2	0.3	16.6	16.0	0.1	16.1	14.3	0.9	15.1	14.3	0.0	14.3	8.1	0.0	8.1	2.0	0.8	2.8	0.0	1.1	1.1	0.0	0.5	0.5
S_{13}	59.0	0.0	59.0	16.8	0.3	17.1	16.4	0.1	16.5	14.9	10.4	25.3	14.9	0.0	14.9	44.1	0.0	44.1	1.9	0.7	2.6	0.0	1.0	1.1	0.0	1.0	1.0
S_{14}	70.5	0.0	70.5	59.4	0.2	59.6	59.0	0.0	59.1	50.3	0.6	50.9	50.3	0.0	50.3	20.2	0.0	20.2	9.1	0.5	9.5	0.1	0.6	0.6	0.0	0.4	0.4
S_{15}	7.1	0.0	7.1	4.5	0.2	4.7	4.4	0.2	4.6	4.2	0.8	5.0	4.2	0.0	4.2	2.9	0.0	2.9	0.2	0.4	0.7	0.0	0.4	0.5	0.0	0.8	0.8
S_{16}	12.9	0.0	12.9	6.3	0.1	6.4	6.2	0.0	6.3	6.2	0.2	6.4	6.2	0.0	6.2	6.7	0.0	6.7	0.2	0.4	0.6	0.0	0.6	0.6	0.0	0.1	0.1
S_{17}	6.0	0.0	6.0	2.8	0.1	2.9	2.7	0.0	2.8	2.5	0.2	2.7	2.5	0.0	2.5	3.5	0.0	3.5	0.3	0.5	0.8	0.0	1.1	1.1	0.0	0.2	0.2
S_{18}	29.8	0.0	29.8	18.4	0.3	18.7	18.3	0.0	18.3	17.7	0.4	18.1	17.7	0.0	17.7	12.1	0.0	12.1	0.7	0.9	1.6	0.1	1.3	1.4	0.0	0.3	0.3
S_{19}	21.1	0.0	21.1	13.6	0.7	14.3	12.6	0.5	13.1	12.5	1.6	14.0	12.5	0.0	12.5	8.6	0.0	8.6	1.2	1.1	2.3	0.0	1.6	1.6	0.0	1.5	1.5
S_{20}	23.9	0.0	23.9	29.0	0.2	29.2	27.3	0.1	27.3	16.9	3.6	20.5	16.9	0.0	16.9	7.0	0.0	7.0	12.1	0.6	12.7	0.0	1.0	1.0	0.0	3.5	3.5
S_{21}	10.1	0.0	10.1	4.5	0.2	4.7	4.4	0.0	4.4	4.2	0.3	4.6	4.2	0.0	4.3	5.9	0.0	5.9	0.2	0.8	1.0	0.0	1.3	1.3	0.0	0.3	0.3
S_{22}	10.6	0.0	10.7	8.7	0.3	9.0	8.4	0.2	8.6	6.0	0.5	6.6	6.0	0.0	6.0	4.6	0.0	4.6	2.7	0.5	3.2	0.0	0.6	0.6	0.0	0.5	0.5
S_{23}	10.3	0.0	10.3	5.4	0.1	5.5	5.3	0.0	5.4	5.3	0.1	5.4	5.3	0.0	5.3	5.0	0.0	5.1	0.1	0.5	0.6	0.0	0.7	0.7	0.0	0.1	0.1
S_{24}	7.3	0.0	7.3	5.2	0.1	5.3	5.1	0.1	5.2	5.0	0.2	5.1	5.0	0.0	5.0	2.3	0.0	2.3	0.2	0.7	1.0	0.0	1.3	1.3	0.0	0.1	0.1
S_{25}	6.7	0.0	6.7	3.2	0.1	3.3	3.1	0.0	3.1	2.8	0.1	2.9	2.8	0.0	2.8	3.8	0.0	3.8	0.3	0.2	0.6	0.0	0.3	0.4	0.0	0.1	0.1
Avg	83.9	0.0	84.0	68.5	0.5	69.0	68.1	0.3	68.4	67.3	1.6	68.9	67.3	0.0	67.3	16.6	0.0	16.6	1.2	2.3	3.5	0.1	7.2	7.3	0.0	1.3	1.3

Table 5: Average execution time of all test cases per build (*Avg. Testing Time*) compared to the average data collection time for all feature groups per build (*Avg. Data Collection Time*) across all subjects. The times are reported in minutes, and the maximum value of each column is shown in bold.

S_{ID}	Avg. Testing Time	Avg. Data Collection Time	Collection/Testing (%)
S_1	26.6	11.7	44
S_2	6.0	1.1	18
S_3	7.0	5.0	71
S_4	6.9	1.7	25
S_5	24.7	2.4	10
S_6	26.3	1.4	5
S_7	17.6	2.1	12
S_8	22.4	0.9	4
S_9	20.7	0.5	3
S_{10}	8.1	0.8	10
S_{11}	6.6	1.2	18
S_{12}	9.5	0.5	5
S_{13}	7.5	1.2	16
S_{14}	7.7	1.4	18
S_{15}	20.7	0.2	1
S_{16}	6.3	0.2	4
S_{17}	13.3	0.1	1
S_{18}	17.6	0.6	3
S_{19}	7.1	0.5	7
S_{20}	17.7	0.7	4
S_{21}	12.3	0.2	2
S_{22}	48.8	0.3	1
S_{23}	68.1	0.2	< 1
S_{24}	13.7	0.2	1
S_{25}	15.5	0.1	1

Further, Table 5 shows the ratio of the total data collection time over the regression testing time for all subjects. The results suggest that data collection time can increase the regression testing time between 1% and 71% with an average of 11% across subjects. As expected, larger systems and test suites tend to correspond to much larger percentages, e.g., 44% for S_1 , 71% for S_3 , and our analysis shows that the percentage of data collection time and subject size (SLOC) are strongly correlated (Spearman’s ρ) with $\rho = 0.79$. This can cause practical issues in a CI context since regression testing should be fast enough to enable the code to be built

and tested several times a day. As a result, this justifies our attempt to investigate whether all feature groups are required to achieve satisfactory TCP effectiveness, especially those groups entailing the largest preprocessing times.

EXP1.1 To answer RQ1.1, we collected data for all builds of each subject and stored the resulting datasets. We recorded the data collection time related to preprocessing and measurement for each feature group, which is used to answer RQ1.1.

RQ1.1 To check whether the differences between the average data collection times of feature groups (ref. Table 4) are statistically significant, we performed multiple pairwise Wilcoxon Signed-rank tests, to compare the data collection times of each feature group pair across the same builds. Note that the Wilcoxon Signed-rank test is a non-parametric test and does not make any distributional assumptions.

As shown in Table 6 in column *p-value*, the pairwise comparison indicates that the differences between all possible feature group pairs are statistically significant. As a result, we can meaningfully rank groups based on their average data collection times. The ranks are depicted in Table 6 with **11**, where a lower rank means higher data collection time. Thus, COD_COV_COM has the highest data collection time, while TES_CHN has the lowest.

We also used the Common Language (CL) effect size analysis to investigate the practical significance of differences. CL is the probability of a randomly sampled item from a population being greater than a randomly sampled item from another population. Table 6 shows the values of effect sizes for each pair across feature groups. The results show that feature groups that rely on coverage analysis (i.e., with COV in their name) have the smallest differences with each other and very large differences with other feature groups. For instance, the difference of COD_COV_COM with other coverage-based feature groups (e.g., F_COV) has a small effect size of around 0.6. Additionally, except for TES_PRO and REC, all feature pairs that do not rely on coverage analysis have large differences with each other (CL

Table 6: Results of pairwise comparison of the data collection time of feature groups using the Wilcoxon Signed-rank test and the Common Language (CL) effect size. Also, each feature group is indicated with a number that shows their rank based on average data collection time per build (higher ranks indicate higher data collection time).

Feature Group Pair		<i>p</i> -value	CL
COD_COV_COM ①	TES_CHN	0.00	1.00
	TES_PRO	0.00	0.98
	REC	0.00	0.95
	TES_COM	0.00	0.80
	F_COV	0.00	0.60
	COD_COV_CHN	0.00	0.59
	COD_COV_PRO	0.00	0.59
	DET_COV	0.00	0.59
COD_COV_PRO ②	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
	DET_COV	0.00	0.51
	COD_COV_CHN	< 0.01	0.50
DET_COV ③	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
	COD_COV_CHN	< 0.01	0.50
COD_COV_CHN ④	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
F_COV ⑤	TES_CHN	0.00	0.98
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.70
TES_COM ⑥	TES_CHN	0.00	0.98
	TES_PRO	0.00	0.93
	REC	0.00	0.88
TES_PRO ⑦	TES_CHN	0.00	0.74
	REC	< 0.01	0.50
REC ⑧	TES_CHN ⑨	0.00	0.73

effect sizes above 0.7).

RQ1.1: The data collection time of feature groups that rely on coverage analysis is significantly higher than other feature groups. Feature groups that rely on test case code analysis have the second-highest, and the feature group that is based on test case execution history has the lowest data collection time.

EXP1.2 We conducted the same experiment as EXP1.1, but by only accounting for data collection time of the features for impacted files to address RQ1.2.

RQ1.2 To investigate the effect of accounting for impacted files in the data collection process, we measured the data collection time of all the features which are based on impacted files. Columns *Avg. Total Collection Time* and *Avg. Impacted Collection Time* in Table 7, show the average data collection time for all features per build and the average data collection time for features based on impacted files per build, respectively, across all subjects. Also, column *Impacted/Total* shows the percentage of the impacted collection time

Table 7: Average data collection time for features related to impacted files per build (*Avg. Impacted Collection Time*) compared to the average data collection time for all feature groups per build (*Avg. Total Collection Time*) across all subjects. The times are reported in minutes, and the maximum value of each column is shown in bold.

<i>S_{ID}</i>	Avg. Total Collection Time	Avg. Impacted Collection Time	Impacted/Total (%)
<i>S₁</i>	11.7	3.5	29
<i>S₂</i>	1.1	0.1	8
<i>S₃</i>	5.0	1.9	38
<i>S₄</i>	1.7	0.5	31
<i>S₅</i>	2.4	0.8	34
<i>S₆</i>	1.4	0.3	25
<i>S₇</i>	2.1	0.7	35
<i>S₈</i>	0.9	0.2	25
<i>S₉</i>	0.5	0.1	26
<i>S₁₀</i>	0.8	0.2	27
<i>S₁₁</i>	1.2	0.4	32
<i>S₁₂</i>	0.5	0.1	21
<i>S₁₃</i>	1.2	0.1	7
<i>S₁₄</i>	1.4	0.4	28
<i>S₁₅</i>	0.2	< 0.1	10
<i>S₁₆</i>	0.2	< 0.1	15
<i>S₁₇</i>	0.1	< 0.1	8
<i>S₁₈</i>	0.6	0.1	22
<i>S₁₉</i>	0.5	0.1	12
<i>S₂₀</i>	0.7	0.1	14
<i>S₂₁</i>	0.2	< 0.1	9
<i>S₂₂</i>	0.3	< 0.1	13
<i>S₂₃</i>	0.2	< 0.1	16
<i>S₂₄</i>	0.2	< 0.1	20
<i>S₂₅</i>	0.1	< 0.1	12

in the total collection time. The collection time of features based on impacted files takes between 7% and 38% of the total data collection time per build across subjects, with an average of 21%. This is a significant portion of the data collection time and, therefore, is a strong justification to investigate whether features that are based on impacted files are required to achieve satisfactory TCP effectiveness. Our analysis also shows that such percentage has a significant correlation (Spearman’s ρ) with both subject size (SLOC, $\rho = 0.56$) and the number of test cases ($\rho = 0.57$), respectively.

RQ1.2: On average, 21% of the total data collection time per build is spent on collecting data for features that are based on impacted files, which is a significant portion of the collection time.

EXP1.3 To answer RQ1.3, we used Spearman’s rank correlation to assess the relationship between the data collection time of features and subject characteristics. Spearman rank correlation is a non-parametric test without any conditions about the data distribution. It is used when the variables are monotonically related and measured on a scale that is at least ordinal. The variables (i.e., data collection time of features and subject characteristics) are ordinal and in monotonic order as shown in Figure 4. Thus, it is an appropriate choice in our context. We performed the correlation analysis between the data collection time of all features as well as each feature group, which was recorded in EXP1.1, and subject size (SLOC) as well as the number of test cases and builds.

RQ1.3 As shown in Table 4, the collection time of each feature group varies across subjects. To explain such variation, we analyzed correlations between subjects’ characteristics and their feature groups’ collection times. We

Figure 4: Linear correlation graphs between two characteristics of subjects (x-axis) and the total data collection time of all feature groups as well as two individual feature groups (y-axis).

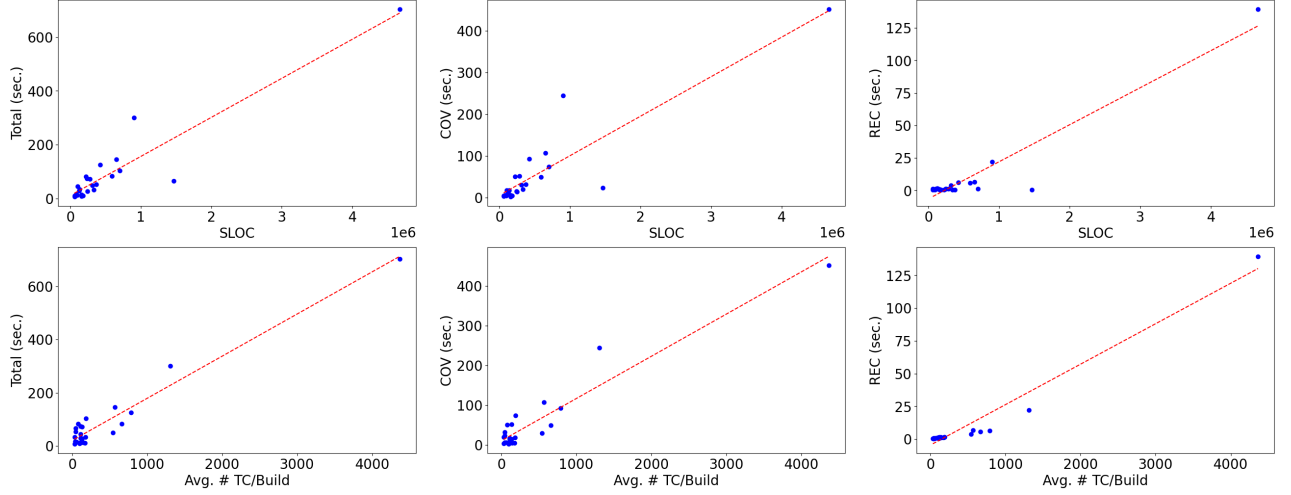


Table 8: Spearman’s rank correlation between characteristics of subjects and data collection time of feature groups. Strong correlations (values greater than 0.8) are shown in bold.

Characteristic	COD_COV_COM	COD_COV_PRO	DET_COV	COD_COV_CHN	F_COV	TES_COM	TES_PRO	REC	TES_CHN	Total
SLOC	0.84	0.81	0.81	0.83	0.83	0.83	0.49	0.41	0.61	0.83
Java SLOC	0.87	0.82	0.82	0.85	0.86	0.86	0.46	0.40	0.57	0.85
# Commits	0.57	0.57	0.57	0.60	0.57	0.51	0.38	0.60	0.76	0.63
# Builds	0.31	0.30	0.30	0.29	0.28	0.18	0.23	0.54	0.44	0.35
# Failed Builds	0.29	0.26	0.26	0.27	0.30	0.24	-0.04	0.36	0.31	0.29
Failure Rate (%)	-0.02	-0.08	-0.08	-0.05	-0.01	0.11	-0.29	-0.06	-0.24	-0.06
Avg. # TC/Build	0.56	0.56	0.56	0.56	0.57	0.40	0.46	0.95	0.53	0.58
Avg. Test Time (min)	-0.18	-0.11	-0.11	-0.15	-0.14	-0.27	0.12	-0.04	-0.00	-0.16

used Spearman’s rank correlation to assess the strength of such correlations as the inspections of scatterplots showed monotonic relationships.

As shown in Table 8, the subjects’ SLOC strongly correlates with most of the feature groups’ data collection times, which is not surprising since the time for coverage and code complexity analysis increases as SLOC increases. We also observe that the average number of test cases per build has a moderate correlation with the data collection time of most feature groups, and a strong correlation (0.95) with REC. REC computes features based on test execution history, and subjects with more test cases have more execution history, this results in higher collection times.

Figure 4 depicts six strong correlations between subjects’ SLOC and data collection times of the feature groups. They suggest there is an increasing monotonic relationship between SLOC and data collection time with a correlation coefficient above 0.8. We can see that there is one outlier in all the scatterplots, which refers to subject S_1 . Thus, we repeated the correlation analysis without such outlier to investigate whether it caused significant inflation in the correlation coefficients. Though the results showed a reduction in correlation coefficients, trends remained consistent.

RQ1.3: The size of the subjects and the average number of test cases per build are positively and significantly correlated with the data collection time of the majority of features.

4.4.2 Training and Testing of Ranking Models for TCP (RQ2)

In the following, we assume that the builds of each subject are assigned a unique id incrementally, according to their time of occurrence.

Model Selection Bertolino et al. [9] showed that Multiple Additive Regression Trees (MART) [33], a.k.a. Gradient boosted regression trees, is the best ML ranking model in the TCP context. Since our study uses a different evaluation metric and different subjects, we conducted an experiment to verify whether or not MART remains the best ML ranking model (M_{opt}) for our subjects as well and thus to obtain the best results. To this end, we evaluated MART against five other ML ranking models, namely LambdaMART (LMART) [34], Random Forest (RF) [16], RankBoost [35], ListNet [36], and Coordinate Ascent (CA) [37]. They were selected as they cover pairwise and listwise ML ranking models, are considered the most popular techniques in the RankLib [38] library, and were already used by Bertolino et al. [9]. For all ranking models, we used an existing implementation found in the RankLib [38] library with the default values for model hyperparameters. For each failed build with id n , we used all feature records from the previous failed builds for training, i.e., all feature records whose build id is between 1 and $n - 1$. We then evaluated the ranking model based on the feature records of build n . We conducted this experiment based on the latest 50 failed builds of each subject. For subjects with less than 50 failed builds (see

Section 4.2), we used all failed builds. We then used the Friedman statistical test [39] to see if there is at least one ML model that is significantly better than others across the same builds and used the Nemenyi post hoc test [40] to single out the best model.

The results of our comparison showed that the RF ranking model performed significantly better than the other models including MART, in contrast to the results reported by Bertolino et al. [9]. Therefore, we decided to use RF for all the experiments throughout the rest of this study. It is worth reminding that the main focus of this study is assessing the impact of test case features on the effectiveness of TCP. Since we expect practitioners to use the best model, we report such results for RF only. A more detailed investigation of the impact of different sets of features for different ML algorithms is out of the scope of this paper.

Hyperparameter Selection Hyperparameter selection can significantly affect the performance of ML models. Thus, we designed an experiment to find the best hyperparameters for the RF ranking model. We investigated the following RF hyperparameters: *rtype*, *srate*, *bag*, *frate*, *tree*, *leaf*, and *shrinkage*. Please refer to the RankLib [38] library for details regarding these hyperparameters. We defined a hyperparameter search space by using the following values for each hyperparameter:

$$rtype \in \{\text{MART}, \text{LMART}\}, srate \in \{0.5, 1.0\},$$

$$bag \in \{150, 300, 600\}, frate \in \{0.15, 0.3, 0.6\}, tree \in \{1, 3, 5\},$$

$$leaf \in \{50, 100, 200\}, shrinkage \in \{0.05, 0.1, 0.2\}$$

The above hyperparameter search space leads to 972 possible combinations. Since exploring all hyperparameter combinations was not computationally possible, we relied on covering arrays [41] to identify 42 combinations with the strength of 3, thus covering all hyperparameter combinations of size 3. We used all the builds (1133 builds) across subjects to evaluate the 42 combinations. Finally, we compared the results of the 42 combinations with the results of the default combination (HC_{def}), which was obtained from the model selection experiment, using the pairwise Wilcoxon Signed-rank test. The results showed that HC_{def} was not the best but was among the best. The best combination (HC_{opt}) achieved an average APFD_C of 0.824 that is higher than that of HC_{def} with 0.813. The difference between HC_{opt} and HC_{def} is statistically significant, though the magnitude of the difference is small. Based on the results, the selected hyperparameter values (HC_{opt}) used through the rest of this study were:

$$rtype = \text{MART}, srate = 0.5, bag = 150, frate = 0.3,$$

$$tree = 5, leaf = 200, shrinkage = 0.2$$

Model and Hyperparameter Selection: The Random Forest (RF) ML ranking model performs significantly better than other investigated ranking models. In addition, the difference between the TCP effectiveness of RF's default and best hyperparameters is small though in favor of the latter.

EXP2.1 To answer RQ2.1, similar to the model selection experiment, we trained RF ranking models for the latest 50 failed builds of each subject using all feature records. Our decision for training the models only based on failed builds is based on the results of a recent study. Via empirical analysis, Elsner et al. [25] showed that training ML-based TCP models by including builds with no failures does not improve the effectiveness of the TCP models. Thus, including such builds only increases the training time without any benefit.

In practice, training ML models for each new build may not be practical or necessary, and we investigate this question in RQ3. However, since our focus is on investigating the impact of features on the effectiveness of ML-based TCP models, we train ranking models for all possible builds across subjects, to analyze results on the largest possible number of builds.

As discussed in Section 4.3, we report APFD_C only based on the failed builds of all subjects. Table 9 shows APFD_C averages and standard deviations of Random Forest ranking models across subjects.

RQ2.1 Column *Full_M* in Table 9 shows APFD_C averages for subjects, when the training data includes the full feature set. Average APFD_C values, for all subjects except the last two, are above 0.7, and the average APFD_C across all builds is 0.82. Thus, it is safe to conclude that including the full feature set in the training of ranking models for test case prioritization leads to promising results for most subjects. However, we can observe a considerable APFD_C variation in Table 9 across subjects (e.g., 0.56 for S_{25} and 0.98 for S_{15}). For this reason, we conducted a correlation analysis to measure potential correlations between APFD_C and subject characteristics. As a result, we did not find any strong or moderate correlations, and therefore we left this question for future work.

RQ2.1: Using the full feature set for training ML ranking models for TCP leads to promising results for most subjects. However, we can observe a considerable variation in results across subjects.

EXP2.2 To answer RQ2.2, we conducted the same experiments as EXP2.1, but we removed features related to impacted files to compare their results with those of the all-features experiment (EXP2.1) and measure their effect on TCP effectiveness.

RQ2.2 Column *IMP_M* in Table 9 reports average APFD_C values for subjects when features related to impacted files are not included in the training data. As shown, the differences between the APFD_C of *Full_M* and *IMP_M* are small across all subjects, and they range between 0.0 and 0.03.

We also ran a Wilcoxon Signed-rank test based on APFD_C results for *Full_M* and *IMP_M* across all builds and subjects. The test results show that there is no statistically significant difference between the APFD_C of *Full_M* and *IMP_M*; *p-value*=0.14. Hence, it is safe to conclude that accounting for the features of impacted files does not bring practical advantages as it does not have a significant effect, across all subjects, on the effectiveness of the ranking models in terms of APFD_C.

Table 9: $APFD_C$ averages and standard deviations across subjects for four different Random Forest ranking models. *Full_M* represents the model that was trained on data that **includes** the full feature set. *IMP_M* was trained on data that **excludes** features related to impacted files. *TES_M*, *REC_M*, and *COV_M* were trained only with features related to test case source code, execution history, and test code coverage, respectively. For each subject (row), the $APFD_C$ of the model with the highest average $APFD_C$ and the lowest standard deviation is shown in bold.

S_{ID}	<i>Full_M</i>	<i>IMP_M</i>	<i>TES_M</i>	<i>REC_M</i>	<i>COV_M</i>
S_{15}	0.98±0.05	0.97±0.05	0.97±0.07	0.98±0.03	0.73±0.24
S_{10}	0.94±0.14	0.94±0.14	0.92±0.17	0.94±0.13	0.66±0.21
S_{14}	0.92±0.16	0.92±0.17	0.92±0.18	0.90±0.20	0.60±0.28
S_7	0.91±0.17	0.91±0.16	0.72±0.23	0.89±0.20	0.46±0.24
S_5	0.90±0.12	0.90±0.12	0.89±0.17	0.88±0.16	0.60±0.17
S_6	0.88±0.23	0.88±0.22	0.76±0.29	0.89±0.22	0.62±0.29
S_{19}	0.88±0.12	0.88±0.13	0.88±0.12	0.87±0.12	0.57±0.25
S_{11}	0.86±0.26	0.86±0.26	0.89±0.21	0.84±0.28	0.62±0.27
S_{23}	0.85±0.15	0.85±0.15	0.85±0.16	0.84±0.16	0.50±0.29
S_{12}	0.85±0.16	0.85±0.16	0.82±0.17	0.85±0.12	0.62±0.22
S_2	0.84±0.04	0.84±0.04	0.85±0.05	0.84±0.04	0.60±0.14
S_9	0.81±0.20	0.81±0.21	0.79±0.24	0.80±0.22	0.52±0.12
S_{18}	0.81±0.24	0.83±0.22	0.86±0.17	0.78±0.28	0.58±0.23
S_4	0.79±0.20	0.79±0.20	0.84±0.19	0.87±0.16	0.53±0.17
S_8	0.78±0.14	0.78±0.15	0.77±0.14	0.68±0.25	0.59±0.23
S_{20}	0.78±0.24	0.78±0.24	0.81±0.21	0.78±0.18	0.66±0.26
S_{16}	0.78±0.22	0.79±0.21	0.77±0.24	0.78±0.23	0.58±0.19
S_1	0.77±0.25	0.78±0.24	0.65±0.25	0.79±0.22	0.42±0.23
S_{21}	0.77±0.23	0.77±0.23	0.80±0.18	0.74±0.24	0.66±0.23
S_{17}	0.77±0.18	0.76±0.18	0.76±0.18	0.73±0.20	0.50±0.12
S_3	0.75±0.25	0.75±0.25	0.68±0.24	0.74±0.26	0.56±0.21
S_{13}	0.73±0.24	0.74±0.24	0.79±0.23	0.66±0.27	0.56±0.22
S_{24}	0.70±0.31	0.67±0.31	0.69±0.29	0.69±0.31	0.60±0.23
S_{22}	0.66±0.29	0.67±0.29	0.71±0.28	0.63±0.28	0.51±0.23
S_{25}	0.56±0.27	0.57±0.28	0.60±0.18	0.54±0.20	0.54±0.09

RQ2.2: Test case features that are related to impacted files do not have a significant effect on the effectiveness of TCP.

EXP2.3 To answer RQ2.3, we conducted two sets of experiments:

- **EXP2.3.1** We trained nine ranking models for each failed build using the same method as in EXP2.1 and EXP2.2. The nine models differ based on the feature groups used for their training since for each model, one feature group was left out.
- **EXP2.3.2** We trained three ranking models for each failed build using the same method as in EXP2.1 and EXP2.2. Again, the three models differ based on the features used for their training. The first model (*COV_M*) was trained using all coverage-related features, all of which are collected via source code coverage analysis of the system under test, i.e., F_COV, COD_COV_COM, COD_COV_PRO, COD_COV_CHN, and DET_COV. The second model (*TES_M*) was trained using features that are calculated via static analysis of test case source code, i.e., TES_COM, TES_PRO, and TES_CHN. The last model (*REC_M*) was trained only based on test case execution history, i.e., REC. The data collection time for features in the same high-level group is heavily driven by data preprocessing of common data sources, and therefore, removing or adding features in such a

Table 10: Pairwise $APFD_C$ comparison results across subjects between models that exclude individual feature groups and the model that includes the full feature set (*Full*). The comparison is done using the Wilcoxon Signed-rank test and the Common Language (CL) effect size. *p-values* below 0.05 are shown in bold.

Experiment Pair		<i>p-value</i>	CL
Full TES_COM		< 0.01	0.51
Full REC		< 0.01	0.54
Full TES_PRO		0.01	0.51
Full COD_COV_COM		0.13	0.50
Full COD_COV_CHN		0.18	0.50
Full DET_COV		0.41	0.50
Full COD_COV_PRO		0.52	0.50
Full F_COV		0.56	0.50
Full TES_CHN		0.97	0.50

Table 11: Pairwise $APFD_C$ comparison results across subjects between models that include individual high-level feature groups and the model that includes the full feature set (*Full*). The comparison is done using the Wilcoxon Signed-rank test and the Common Language (CL) effect size. *p-values* below 0.05 are shown in bold.

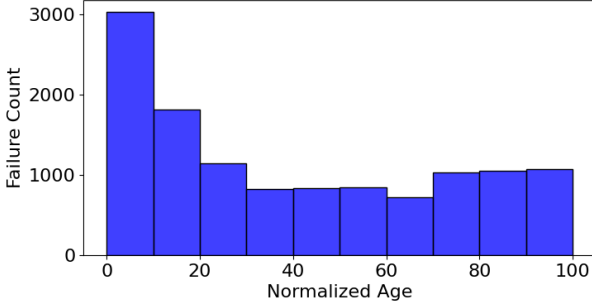
Experiment Pair		<i>p-value</i>	CL
Full COV_M		< 0.01	0.79
Full REC_M		< 0.01	0.51
Full TES_M		< 0.01	0.53

group does not make a practical difference. Therefore, understanding the impact of each high-level group can help practitioners select the best feature group(s) when the collection of all features is not possible.

As we explained in EXP2.3.2 above, the feature extraction cost of a single feature (group) within a high-level group is nearly the same as the cost of extracting all features in that group. As a complementary lower-level analysis, that is admittedly of less practical importance, we investigated the impact of removing one (low-level) feature group and the importance of each individual features in RQ2.4. Given the large volume of our data and the high execution time of the experiments when training Random Forest models, we could not train these models using all possible feature (group) combinations.

RQ2.3 The results of EXP2.3.1 show that excluding any one of the feature groups does not cause a significant decrease in effectiveness for the trained models. We conducted nine Wilcoxon Signed-rank tests on $APFD_C$ results for all builds across subjects. As shown in Table 10, among all nine feature groups, removing TES_COM, REC, or TES_PRO causes a statistically significant difference in $APFD_C$ with *Full_M*; TES_COM: *p-value*<0.01, REC: *p-value*<0.01, and TES_PRO: *p-value*=0.01. However, the CL effect size analysis shows that the magnitude of their impact is practically negligible (CL is around 0.5). Based on the above results, we can conclude that removing any of the nine feature groups does not have a practical impact on the $APFD_C$ of the ranking models.

Figure 5: The histogram shows the relationship between the age of all test cases and their failure frequency. The x-axis shows the normalized age between 0 and 100 and the y-axis shows the number of failures of all test cases in a given age period.



Columns TES_M , REC_M , and COV_M in Table 9 report the average $APFD_C$ of ranking models trained with features related to test case source code, execution history, and test code coverage, respectively. Though the $APFD_C$ of COV_M is low, REC_M reaches similar $APFD_C$ to $Full_M$. To analyze the result of EXP2.3.2, we conducted three Wilcoxon Signed-rank tests to assess the differences of the three models with $Full_M$. As shown in Table 11, the $APFD_C$ results of all three models are statistically different from $Full_M$; $p\text{-value} \leq 0.01$. The CL effect size analysis for $APFD_C$ results shows that the difference with $Full_M$ for COV_M , with a value of 0.79, is practically significant, as opposed to the CL effect sizes of TES_M and REC_M , which are 0.53 and 0.51, respectively. Thus we can conclude that, among all high-level feature groups, TES_M —which is trained with features related to the source code of test cases— and REC_M —which is trained with features related to the execution history of test cases— achieve the best results, with nearly the same $APFD_C$ as the models trained with the full feature set. Further, the models trained with only the coverage-based feature groups achieve the worst $APFD_C$ and are not reliable individually.

RQ2.3: Each of the nine feature groups does not have a significant impact on TCP effectiveness, individually. This is different for high-level feature groups. Features related to test case source code or execution history have nearly the same impact on TCP effectiveness as all features, whereas coverage-based features have a significantly lower impact.

EXP2.4 To address RQ2.4, we rely on RankLib [38] as it provides model feature statistics for the RF model. It captures how frequently each feature is used to split nodes of trained regression trees. Such decision trees use information gain, which is based on the concept of entropy, to determine which feature to use to further split the tree at each training step. Therefore, a higher usage frequency in the tree for a feature indicates that it tended to yield higher information gains at training time. We use feature usage frequencies to measure the impact of individual features in our feature groups.

RQ2.4 Table 12 reports the average usage frequency of

Table 12: Top 15 frequently used individual features in models that were trained on the full feature set. The *Avg. Freq.* column shows the average usage frequency of a feature across all ranking models.

Feature Group	Feature	Avg. Freq.
REC	Age	17034
TES_PRO	OwnersExperience	10618
TES_PRO	AllCommittersExperience	7643
REC	TotalMaxExeTime	5409
REC	LastExeTime	4095
TES_PRO	OwnersContribution	3997
TES_PRO	CommitCount	3986
REC	TotalAvgExeTime	3977
REC	RecentAvgExeTime	3850
REC	RecentMaxExeTime	3741
TES_COM	RatioCommentToCode	3695
TES_COM	CountStmtDecl	3383
TES_COM	CountLineCodeDecl	3342
TES_COM	CountLineBlank	3149
TES_COM	CountStmtExe	2825

the top 15 features in $Full_M$ for models across all subjects. Top features belong to the three feature groups REC, TES_PRO, and TES_COM, all of which are identified as significant feature groups by RQ2.3.

From the TES_COM group, features capturing the size of test cases show higher usage, which is to be expected as large test cases tend to have a higher execution time and coverage. Also, from the TES_PRO group, features capturing the experience of test case developers and the number of commits are more important. We conjecture that experienced developers may better understand the system under test to develop high-quality test cases compared to less experienced developers. Also, higher numbers of commits for a test case suggest that the test case is under active development and is being updated to detect faults.

From the REC feature group, features related to previous execution times and the age of test cases are the most frequently used. The former are good indicators of future test case execution times and explain why execution time-related features have high usage. However, seeing the age of a test case as the most important among all features may seem surprising at first glance. To explain this, we analyzed the relation between the age of test cases and their failure occurrences across all subjects. As shown in Figure 5, failures drop sharply after the first 10% and 20% of the builds on which they are executed. Thus, the age of a test case is a good predictor of its failure probability. Further, this suggests that all subjects contain obsolete test cases that are not actively maintained and not kept updated. When a new functionality of the SUT is developed, a number of test cases are developed to test it, and once the functionality works as expected and becomes stable over time, their corresponding test cases often pass across builds.

RQ2.4: The most frequently used test case features in the ML ranking models belong to REC, TES_PRO, and TES_COM feature groups. These features include test case size, development history, failure history, and age.

EXP2.5 A heuristic-based TCP approach prioritizes test cases based on single features of test cases [25]. Given a test suite T and a test case feature f_i , we define two heuristic-based ranking models $M_{f_i,a}$ and $M_{f_i,d}$. $M_{f_i,a}$ produces a test case ordering T_a^* in which test cases are sorted by their corresponding value of f_i in ascending order. Conversely, $M_{f_i,d}$ produces T_d^* in which test cases are sorted in descending order.

Elsner et al. [25] reported that heuristic-based TCP models that are based on $F_MaxTestFileFailRate$ (defined in 3.2.2) outperform other heuristics. We first investigated if the same results held with the subjects of this study. To do that, for each test case feature f_i ($1 \leq i \leq 150$), we applied the $M_{f_i,a}$ and $M_{f_i,d}$ heuristic-based models (300 models in total) and compared their results. We observed that the $M_{f_{56},d}$ model, which is based on the $F_FailRate(Total)$ feature (Section 3.2.2), performed best for the majority of subjects, in contrast to previous work [25]. Elsner et al. [25] also concluded that heuristic-based TCP models are better than ML-based TCP models in terms of average APFD_C, though not statistically different. Thus, in this experiment, we compared the results of our ML-based TCP model, RF, with the best heuristic, i.e., $F_FailRate(Total)$.

RQ2.5 To compare the effectiveness of the best H_M (heuristic-based) model with the $Full_M$ ML-based model, we conducted a Wilcoxon Signed-rank test for all builds across subjects. The test results showed that the difference between the two models is statistically significant ($p\text{-value} < 0.01$), and the $Full_M$ model, with an average APFD_C of 0.82, clearly outperforms the H_M model, with an average APFD_C of 0.71.

For further analysis, we compared H_M and $Full_M$ by conducting Wilcoxon Signed-rank tests for each subject. Table 13 shows the statistical test results as well as APFD_C averages and standard deviations across subjects. Based on Table 13, H_M outperformed $Full_M$ in only four subjects in terms of average APFD_C, and in only one of these four subjects, the difference was statistically significant. For the rest of the subjects, $Full_M$ achieved a higher average APFD_C, and in 13 subjects, the difference was statistically significant.

Further, using Spearman’s rank correlation coefficient (ρ), we investigated the relationship between subject characteristics and their $Full_M$ and H_M average APFD_C. The analysis showed that a moderate correlation, with $\rho = -0.28$, exists between subject failure rate and the difference between $Full_M$ and H_M in terms of average APFD_C. Thus, the H_M model tends to perform better than $Full_M$ in subjects that have higher failure rates, especially in the presence of a large number of failed builds (e.g., S_4), which can be explained since H_M is based on the failure rate heuristic. However, Beller et al. [42] conducted a comprehensive analysis of TravisCI projects and showed that for all 1,108 Java projects with test executions, the ratio of builds with at least one failed test case has a median of 2.9% and a mean of 10.3%. Therefore, since high failure rates (e.g., failure rates above 30% in our subjects) are not common in CI contexts, H_M is probably not a good option in practice.

To conclude, our empirical analysis shows that the Random Forest TCP ranking model, when trained on the full

Table 13: Pairwise APFD_C comparison across subjects between the heuristic model (H_M) based on $F_FailRate(Total)$ and ML-based models that were trained on the full feature set ($Full_M$). The comparison is done using the Wilcoxon Signed-rank test and the Common Language (CL) effect size. For each subject (row), the APFD_C of the model with the higher average APFD_C and the lower standard deviation is shown in bold. Also, $p\text{-values}$ below 0.05 are shown in bold.

S_{ID}	$Full_M$	H_M	$p\text{-value}$	CL
S_2	0.84±0.04	0.82±0.05	< 0.01	0.86
S_{16}	0.78±0.22	0.58±0.16	< 0.01	0.78
S_{19}	0.88±0.12	0.50±0.38	< 0.01	0.74
S_{21}	0.77±0.23	0.49±0.33	< 0.01	0.73
S_6	0.88±0.23	0.68±0.30	< 0.01	0.72
S_{18}	0.81±0.24	0.60±0.33	< 0.01	0.69
S_{20}	0.78±0.24	0.57±0.32	< 0.01	0.68
S_{15}	0.98±0.05	0.86±0.30	< 0.01	0.66
S_1	0.77±0.25	0.58±0.32	< 0.01	0.66
S_7	0.91±0.17	0.84±0.25	0.13	0.65
S_3	0.75±0.25	0.54±0.34	< 0.01	0.64
S_{17}	0.77±0.18	0.69±0.23	0.07	0.62
S_9	0.81±0.20	0.83±0.12	0.30	0.62
S_{13}	0.73±0.24	0.64±0.29	0.43	0.61
S_{12}	0.85±0.16	0.77±0.21	0.02	0.60
S_5	0.90±0.12	0.74±0.34	0.01	0.59
S_{22}	0.66±0.29	0.55±0.35	0.08	0.58
S_8	0.78±0.14	0.71±0.18	0.07	0.56
S_{14}	0.92±0.16	0.84±0.28	< 0.01	0.56
S_{11}	0.86±0.26	0.86±0.21	0.39	0.55
S_{24}	0.70±0.31	0.66±0.32	0.44	0.54
S_{25}	0.56±0.27	0.59±0.28	0.81	0.48
S_{10}	0.94±0.14	0.94±0.22	0.53	0.46
S_{23}	0.85±0.15	0.85±0.22	0.27	0.45
S_4	0.79±0.20	0.89±0.16	0.04	0.42

feature set, performs overall significantly better than the best heuristic-based model based on the $F_FailRate(Total)$ feature. Nevertheless, the heuristic-based model performs significantly better for a few subjects. Though this remains to be confirmed by further studies, an analysis of the characteristics of these projects suggests this is due to a combination of large numbers of failed builds with high failure rates, which are unlikely in a CI context.

RQ2.5: The Random Forest ranking model, when trained on the full feature set, significantly outperforms the best heuristic-based model for most subjects. However, the heuristic-based model performs significantly better for a few subjects, all of which have high build failure rates.

4.4.3 Effectiveness Decay of ML-Based TCP Models (RQ3)

EXP3 To answer RQ3, we used all of the pretrained models from EXP2.1 and tested each model on the subsequent builds using features that were computed based on preprocessed data (i.e., dependency graph, source code, and process metrics) of the build related to the model. Assume we have n builds ($B = \{b_1, b_2, \dots, b_n\}$) and n pretrained models ($M = \{m_1, m_2, \dots, m_n\}$), and the pretrained model of b_i is m_i where $1 \leq i \leq n$. In this experiment, we used

Table 14: An example of retraining window (RW) values when the number of builds is equal to 5. Each cell shows the RW when TCP is applied to the build of the cell’s row using the pretrained model of the cell’s column. For instance, if we use m_1 for b_4 , then $RW = 3$.

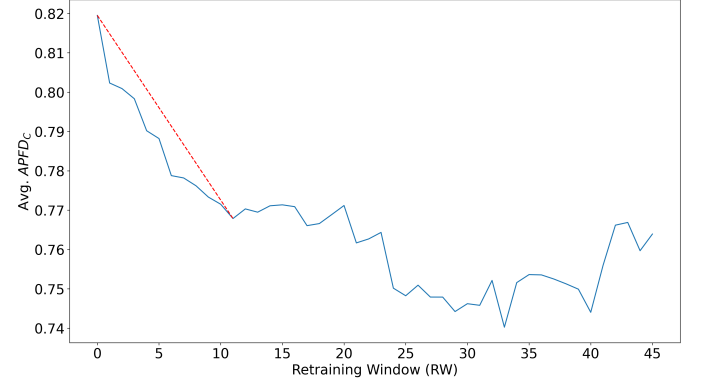
	m_1	m_2	m_3	m_4	m_5
b_1	0	-	-	-	-
b_2	1	0	-	-	-
b_3	2	1	0	-	-
b_4	3	2	1	0	-
b_5	4	3	2	1	0

b_k and m_k ($1 \leq k \leq n$ and $n = 50$ for the latest 50 failed builds) from EXP2.1 and all its relevant preprocessed data. Then, for each b_i after b_k , i.e., $\{b_{k+1}, b_{k+2}, \dots, b_n\}$, we recomputed its test case features based on the preprocessed data available in b_k , ranked its test cases using m_k , and evaluated the ranking. The main motivation for reusing pretrained ranking models is to reduce the cost of data preprocessing and model training, and for this reason, we wanted to assess the impact of such reuse on $APFD_C$ decay. When the preprocessed data of b_k did not contain enough information to compute some of the test case features in b_i (i.e., new test cases or new code coverage graphs), we used the mean substitution imputation method for missing values, as it is a simple, efficient, and effective method [43], and assigned those missing features with the average value across all other test cases with such feature values.

Results As we discussed in EXP3, we have tested the pretrained ranking model for each failed build on its following failed builds to measure how the $APFD_C$ of the pretrained models decreases over time. Here we assume a retraining window (RW) as the distance (i.e., number of builds) between the builds based on which the model is trained and tested, respectively, e.g., when m_k , that is trained based on the data captured up to b_k , is used for TCP in builds up to b_{k+4} without retraining, then $RW = 4$. Table 14 shows an example of RWs when the number of builds is equal to 5. Each cell with row b_i and column m_j in Table 14 is equal to the RW when the pretrained model m_j is used to prioritize the test cases of b_i .

We computed the $APFD_C$ for all valid builds and pretrained models across subjects. Figure 6 shows the $APFD_C$ of pretrained models, based on all builds across subjects, for RWs ranging from 0 to 45. The x-axis is the RW, and the y-axis is the average $APFD_C$ of all pretrained models with the same RW. As shown in Figure 6, there is a steady decreasing $APFD_C$ trend until RW is about 11. The two points $RW = 0$ and $RW = 11$ are connected with a red dotted line in Figure 6. The slope of this line, which is the average decrease in $APFD_C$ after each RW increment, is equal to -0.005. However, $APFD_C$ values above $RW = 11$ display unstable trends. Hence, we can conclude that retraining ranking models with a RW of less than 11 builds is necessary to obtain predictable results across all subjects. However, given the available budget for data collection and model retraining, we also conclude that ML ranking models should be retrained as frequently as possible to achieve the best possible TCP effectiveness.

Figure 6: This graph shows the relationship between the retraining window (RW) of ranking models across all subjects and their average $APFD_C$.



RQ3: Retraining ranking models with a retraining window (RW) of less than 11 builds is necessary to obtain predictable results. Also, the RW should be as small as possible to obtain the best TCP effectiveness.

4.4.4 Trade-off Between Data Collection Time and TCP Effectiveness for Features (RQ4)

Based on the results of RQ1-3, three different decisions can be taken concerning the application of TCP techniques for a software system to achieve the best trade-off, in a given context, between data collection overhead and TCP effectiveness. In the following, we provide guidelines regarding such decisions.

Use ML model relying on the full feature set with retraining at each build or every 11 builds. When higher TCP effectiveness is essential, and the cost of data collection can be afforded, the use of the full feature set for training ML-based TCP models is recommended since, as shown in RQ2, it yields the best results. If necessary, based on the results of RQ3, to decrease data collection time at the cost of a slight effectiveness loss, model retraining can be conducted every 11 builds instead of each build.

Use ML model relying on the REC feature set. When the overhead of the full feature set cannot be afforded, regardless of the training strategy, and when a small decrease in model effectiveness is acceptable given a much lower data collection time, an ML model relying on the REC feature set is recommended. As discussed in RQ1, the data collection time of the REC feature group (i.e., features based on test execution history) is in the order of seconds and negligible. Further, RQ2 results show that not only removing the REC feature group causes statistically significant differences in model effectiveness, but furthermore training the model with only the REC features achieves results that are close to those obtained with the full feature set. Finally, our comparison between ML models based on REC features and heuristics models shows that the former’s effectiveness is higher than the latter’s.

Use failure-based heuristics. When no data collection overhead is acceptable, the use of failure-based heuristics

can be effective, especially in the context of systems with a high number of builds and high failure rates, thus potentially enabling the definition of effective heuristics.

RQ4: To achieve the best TCP effectiveness when data collection cost is affordable, we recommend using Random Forest (RF) ranking models that are trained on the full feature set. When the data collection is not affordable, we suggest an RF model relying on the REC feature set. Finally, the fastest and least effective approach is the use of failure-based heuristics.

4.4.5 Configuration

Data collection for each subject was conducted using one CPU core (Intel Xeon Gold 6234 CPU @ 3.30GHz) with a memory usage ranging between 1GB to 60GB, depending on the size of the subjects. Also, our data collection tool, all results, and datasets are made publicly available³. We used PyDriller [44] for mining git repositories, Understand [21] for static and dependency analysis of source code, and RankLib [38] for training our TCP machine learning ranking models.

4.5 Threats to Validity

4.5.1 Internal Threats

As explained in Section 2.2, for scalability reasons invoked by the industry partner supporting this research, the test case code coverage that we computed is an estimation based on static code coverage, which is computed using Understand [21], and the history of file changes in the system's code repository. This, however, can lead to overestimating test case code coverage. Also, based on Section 3.2, our proposed test case coverage is at the file level, which can further worsen the overestimation issue. Future work should systematically investigate trade-offs between finer-grained coverage measurement, scalability, and TCP effectiveness.

In Travis CI, each CI cycle (build) includes one or more jobs, each of which tests the source code for different configurations and platforms (e.g., a project may have two jobs to test the code for Java 1.6 and 1.8). In most cases, the jobs share the same test cases (98% of test cases for 24 out of 25 of our subjects are the same across jobs of the same build), resulting in more than one execution record for the repeated test cases that affect the quality of the datasets negatively. To mitigate this issue, we only focused on a job with the highest number of test cases. This reduces our data sets size in terms of the number of test cases around 2% that may have a negligible effect on our experiments since the large number of builds that are used in the experiments compensate for this reduction. An alternative way to deal with this issue is to include extra features concerning the platform. However, we left this to future work.

Also, as discussed in Section 4.4.2, we removed frequent-failing test cases using the three-sigma rule technique that may deflate the results of the ML models. To investigate this, we measured the difference between the effectiveness of our ML models with/without outlier test cases that showed

there is no statistically significant difference between the two cases, i.e., removing outlier test cases does not deflate the results of the ML models.

4.5.2 External Threats

We relied on *GHTorrent* [27] to search for open-source projects. In addition, *TravisTorrent* [23] was used to fetch build logs from Travis CI and to find the commits that are related to builds. We also included a subset of test case execution record datasets that were published by *RTPTorrent* [22]. There might be flaws in these tools and datasets—though it is unlikely given how widely used they are—that impact the quality of our data collection.

5 RELATED WORK

Many studies have addressed Test Case Prioritization (TCP) in the regression testing context. Overall, these studies can be classified into two groups: heuristic-based and ML-based techniques. Since our work is ML-based, in the following, we only briefly discuss the heuristic-based group.

Heuristic-based TCP. The studies in this group apply heuristics to TCP that are mainly defined based on test coverage and execution history.

Coverage-based heuristics are based on the principle that increasing structural test coverage of the System Under Test (SUT) increases the chances of fault detection [45]. Structural coverage measures include statement coverage [30], functions/methods coverage [46], and modified condition/decision coverage [47]. Overall, coverage-based heuristics can be grouped into two sub-groups: total coverage and additional coverage [48]. At each prioritization step, the former selects the test case with the highest coverage, while the latter selects the test case with the highest coverage of entities not yet covered by higher-priority test cases.

Extracting precise coverage information at a reasonable cost is the major drawback of coverage-based heuristics. Coverage information can be collected either by static or dynamic analysis. Static analysis techniques are more easily amenable to collecting coverage and conducting impact analysis. However, they overestimate the coverage and impact of test cases, and relying solely on them may undermine the effectiveness of TCP. Dynamic analysis is the most precise technique and requires the execution of tests on the instrumented (i.e., binary or source code instrumentation) version of the SUT. Dynamic analysis techniques are platform-dependent, time-consuming for large codebases, and may not be applicable for real-time systems because code instrumentation may cause timeouts or interrupt normal test execution. Overall, dynamic analysis techniques are difficult or even impossible to apply in practice, specifically in the CI context, mainly due to computational overhead and applicability [1], [2], [3], [4], [5]. Thus, our work only investigates the use of static coverage information. We conjecture that mixing static coverage information with other relevant information, such as test cases' execution history, can compensate for the static analysis techniques' tendency to overestimate.

Heuristics that are based on test case execution history assume that previously failed test cases are more likely to fail again. For example, Kim and Porter [45] proposed an execution history heuristic that calculates ranking scores

3. <https://github.com/Ahmadreza-SY/TCP-CI>

based on the average failure rate of test cases. The execution history is easily accessible, and some studies such as [2], [25] reported that the effectiveness of execution history heuristics is almost the same as ML-based TCP. However, as discussed by Pan et al. [8], the reported training of ML models is problematic due to (1) limited feature sets, (2) unbalanced datasets containing only a small number of failed test cases, and (3) suboptimal ML techniques in the TCP context. To further investigate, we performed a comparison between the best heuristics based on the failure history of test cases and the best ML models, as discussed in Section 4.

ML-based TCP: Work in this group trains ML models based on features collected from different sources, such as test execution history, to prioritize test cases. Compared to heuristics that are often defined based on a single feature, ML models can be trained on a set of features from different sources. This enables ML models to deal with the complexities of the TCP problem for complex systems in the CI context. Also, ML techniques either support online training or can be retrained to account for the dynamic nature of CI [49]. However, heuristics are static, and there is no standard procedure to tune them based on new changes.

Pan et al. [8] surveyed ML-based TCP techniques and reported that existing ML-based papers investigated a wide variety of ML techniques including reinforcement learning [2], [9], [4], [3], [50], [51], [52], clustering [53], [54], [55], [56], [57], [58], [59], ranking models [9], [24], [32], [60], [61], [62], [63], [64], [65], [66], [67], [68], [69], and natural language processing [56], [24], [62], [70], [71]. However, each study only used a small number of features that are either easily collected or publicly published by existing work. Also, most existing works evaluated the proposed TCP techniques based on subjects with very low numbers of failed test cases (e.g., 2% failure rate across subjects in [9]) and short regression testing time (average regression testing below 90 seconds). Having a sufficient number of failed builds is important for evaluating such TCP techniques and creating the balanced datasets required by many ML techniques. Further, applying TCP techniques to systems whose regression testing takes a short time is not practically beneficial and therefore not representative in an experimental context. Our study focuses on these three mentioned issues by investigating the use of a comprehensive feature set for training ML TCP models based on 25 realistic subjects whose test failure rates have a median of 14% and their regression testing time takes at least 5 minutes with a median of 12 minutes. In the following, we discuss the three studies closest to ours.

Bertolino et al. [9] analyzed the performance of ten ML algorithms, including three RL algorithms, for test prioritization in CI. Through experimental analysis, they showed that RL-based approaches to test case prioritization can adapt to new changes as their TCP effectiveness is less affected by changes. They also reported that MART [38], a pairwise ranking model that is trained using an ensemble of boosted regression trees, reaches the highest effectiveness among the ten ML and RL techniques. However, their study suffers from the issues discussed above. More specifically, they only use limited features that are defined based on the execution history of test cases and their complexities. They also evaluated their work only based on six subjects, all of

which include very low numbers of failed test cases (only 49 failed cycles among 2.6k builds across all subjects) and short regression testing time (maximum of 22 seconds across subjects).

In a similar work to the previous one, Bagherzadeh et al. [6] conducted a thorough analysis of ten state-of-the-art deep reinforcement learning techniques in the TCP context. Through extensive empirical analysis, they showed that the best result from RL techniques can reach similar effectiveness as MART in the TCP context. This study is based on the same data set as that of Bertolino et al. [9]. Thus it suffers from the same issues (limited number of features and subjects) as discussed above.

Elsner et al. [25], investigated a more comprehensive set of features for training ML models in the TCP context. They found that features from test execution history work better than other types of features. They also showed that well-known and simple heuristics based on the failure rate of test cases often outperform complex ML models, which was unexpected. While the paper used a relatively large number of subjects with an adequate number of failed test cases, they only used subjects containing only 16 features and thus this study is far from being comprehensive. Indeed, it missed many essential features, including features based on static coverage analysis and the code complexity of test cases and the SUT. Our study shows that complexity features play an important role in training highly effective ML models for TCP. They also did not investigate the data collection time associated with features, which is necessary to perform trade-off analysis on the cost and benefits of features. Finally, they failed to use the state-of-the-art ML models in a TCP context (MART) as reported by previous studies [9]. Instead, they trained a point-wise ranking model that, according to existing studies (Bertolino et al. [9] and Bagherzadeh et al. [6]), provides lower effectiveness compared to pairwise ranking models such as MART. We believe that all these elements explain why the effectiveness of their ML models is lower than simple heuristics. We further investigated (Section 4) this issue by comparing the results of MART models against the heuristics based on failure rate, which reaches the best effectiveness as reported by Elsner et al. [25]. The results show that the models trained either based on the full feature set or features related to the execution history outperform the heuristic model.

Overall, our work complements existing ML-based TCP studies by performing a comprehensive investigation of features (150), obtained from various sources, for training ML TCP models. This allows us to provide practical insights on the benefits and costs of ML-based techniques through extensive empirical analysis. Also, we provide a tool and a set of diverse subjects that can be used as a benchmark for future studies.

6 CONCLUSION

In this work, focused on ML-based Test Case Prioritization (TCP), we have carefully defined a comprehensive set of features aimed at predicting the probability of regression failure, based on related studies, and a data model that captures entities and their relations in a typical CI environment. The features are categorized according to common data collection sources and steps into three high-level groups:

REC containing features capturing the test execution history, TES containing features characterizing the complexity of the test case source code, their changes, and their development process, and COV containing features that capture test case code coverage of changes in the system under test (SUT). We have developed a tool to collect these features for 25 carefully selected open-source projects in order to carry out an extensive experimental study to analyze the cost and benefits of the feature groups for ML-based TCP. The ultimate goal is to provide concrete and precise recommendations regarding what data to collect to effectively support TCP in a scalable way.

The results of our study show that:

- The data collection time of all features ranges between 0.1 to 11.7 minutes across subjects for each build. Also, the trained models for TCP based on the full feature set can achieve promising results in terms of APFD_C across most subjects, with an average of 0.82.
- Coverage-related features (i.e., feature groups with COV in their name) are the most expensive features to collect, while they have the least impact on the effectiveness of ML TCP models.
- REC features are the least expensive to collect, while models relying on them achieve an APFD_C close to models trained with the full set of features.
- Not retraining models for more than 11 builds leads to unstable APFD_C. Also, to achieve the best TCP effectiveness, the models should be retrained as frequently as possible.

Access to high-quality datasets is one of the main issues for devising and evaluating new TCP techniques. Thus, we also made our datasets publicly available, which can serve as a benchmark for future studies. In addition, we plan to extend our data collection tool to support the data collection at the method level and investigate how using method-level data can impact the TCP techniques in terms of data collection cost and effectiveness. Further, extending the data collection and analysis to other CI tools such as GitHub Actions and other programming languages would be a useful endeavor.

ACKNOWLEDGEMENT

This work was supported by a research grant from Huawei Technologies Canada, Mitacs Canada, as well as the Canada Research Chair and Discovery Grant programs of the Natural Sciences and Engineering Research Council of Canada (NSERC). This research was also enabled in part by the computation support provided by Compute Canada⁴.

REFERENCES

- [1] S. Elbaum, G. Rothermel, and J. Penix, "Techniques for improving regression testing in continuous integration development environments," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 235–245.
- [2] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017, pp. 12–22.
- [3] J. A. P. Lima, W. D. Mendonça, S. R. Vergilio, and W. K. Assunção, "Learning-based prioritization of test cases in continuous integration of highly-configurable software," in *Proceedings of the 24th ACM Conference on Systems and Software Product Line: Volume A-Volume A*, 2020, pp. 1–11.
- [4] J. A. do Prado Lima and S. R. Vergilio, "A multi-armed bandit approach for test case prioritization in continuous integration environments," *IEEE Transactions on Software Engineering*, 2020.
- [5] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.
- [6] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Transactions on Software Engineering*, pp. 1–1, 2021.
- [7] A. Gepperth and B. Hammer, "Incremental learning algorithms and applications," in *European Symposium on Artificial Neural Networks (ESANN)*, 2016.
- [8] R. Pan, M. Bagherzadeh, T. A. Ghaleb, and L. Briand, "Test case selection and prioritization using machine learning: A systematic literature review," *arXiv e-prints*, pp. arXiv–2106, 2021.
- [9] A. Bertolino, A. Guerriero, B. Miranda, R. Pietrantuono, and S. Russo, "Learning-to-rank vs ranking-to-learn: Strategies for regression testing in continuous integration," in *In 42nd International Conference on Software Engineering (ICSE)*, 2020.
- [10] Mockus and Votta, "Identifying reasons for software changes using historic databases," in *Proceedings 2000 International Conference on Software Maintenance*, 2000, pp. 120–130.
- [11] A. Hindle, D. M. German, M. W. Godfrey, and R. C. Holt, "Automatic classification of large changes into maintenance categories," in *2009 IEEE 17th International Conference on Program Comprehension*, 2009, pp. 30–39.
- [12] S. Levin and A. Yehudai, "Boosting automatic commit classification into maintenance activities by utilizing source code changes," in *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering*, ser. PROMISE. New York, NY, USA: Association for Computing Machinery, 2017, p. 97–106. [Online]. Available: <https://doi-org.proxy.bib.uottawa.ca/10.1145/3127005.3127016>
- [13] S. Zafar, M. Z. Malik, and G. S. Walia, "Towards standardizing and improving classification of bug-fix commits," in *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2019, pp. 1–6.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://www.aclweb.org/anthology/N19-1423>
- [15] Y. Zhu, R. Kiros, R. Zemel, R. Salakhutdinov, R. Urtasun, A. Torralba, and S. Fidler, "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books," in *2015 IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 19–27.
- [16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [17] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sep 1995. [Online]. Available: <https://doi.org/10.1007/BF00994018>
- [18] Wikipedia, "Tf-idf — Wikipedia, the free encyclopedia," <https://en.wikipedia.org/wiki/Tf-idf\textendashidf>, 2021.
- [19] E. D. Berger, C. Hollenbeck, P. Maj, O. Vitek, and J. Vitek, "On the impact of programming languages on code quality: A reproduction study," *ACM Trans. Program. Lang. Syst.*, vol. 41, no. 4, Oct. 2019. [Online]. Available: <https://doi-org.proxy.bib.uottawa.ca/10.1145/3340571>
- [20] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: <https://doi-org.proxy.bib.uottawa.ca/10.1145/2939672.2939785>

4. www.compute-canada.ca

- [21] Scientific Toolworks, Inc., "Understand™," <https://www.scitools.com/>, 2020, retrieved October 14, 2020.
- [22] T. Mattis, P. Rein, F. Dürsch, and R. Hirschfeld, "Rtptorrent: An open-source dataset for evaluating regression test prioritization," in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 385–396. [Online]. Available: <https://doi-org.proxy.bib.uottawa.ca/10.1145/3379597.3387458>
- [23] M. Beller, G. Gousios, and A. Zaidman, "Travis ci and github for full-stack research on continuous integration," in *Proceedings of the 14th working conference on mining software repositories*, 2017.
- [24] B. Busjaeger and T. Xie, "Learning for test prioritization: an industrial case study," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 975–980.
- [25] D. Elsner, F. Hauer, A. Pretschner, and S. Reimer, "Empirically evaluating readily available information for regression test optimization in continuous integration," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 491–504.
- [26] M. di Biase, A. Rastogi, M. Bruntink, and A. van Deursen, "The delta maintainability model: Measuring maintainability of fine-grained code changes," in *Proceedings of the Second International Conference on Technical Debt*, ser. TechDebt '19. IEEE Press, 2019, p. 113–122. [Online]. Available: <https://doi.org/10.1109/TechDebt.2019.00030>
- [27] G. Gousios, "The ghtorrent dataset and tool suite," in *Proceedings of the 10th Working Conference on Mining Software Repositories*, ser. MSR '13. Piscataway, NJ, USA: IEEE Press, 2013, pp. 233–236. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2487085.2487132>
- [28] Wikipedia contributors, "68–95–99.7 rule — Wikipedia, the free encyclopedia." [Online]. Available: https://en.wikipedia.org/wiki/68%E2%80%959395%E2%80%9599.7_rule
- [29] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001*. IEEE, 2001, pp. 329–338.
- [30] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No. 99CB36360)*, 1999, pp. 179–188.
- [31] Q. Peng, A. Shi, and L. Zhang, "Empirically revisiting and enhancing ir-based test-case prioritization," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 324–336. [Online]. Available: <https://doi.org/10.1145/3395363.3397383>
- [32] J. Chen, Y. Lou, L. Zhang, J. Zhou, X. Wang, D. Hao, and L. Zhang, "Optimizing test prioritization via test distribution analysis," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 656–667.
- [33] J. H. Friedman, "Greedy function approximation: A gradient boosting machine." *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. [Online]. Available: <https://doi.org/10.1214/aos/1013203451>
- [34] Q. Wu, C. J. C. Burges, K. M. Svore, and J. Gao, "Adapting boosting for information retrieval measures," *Information Retrieval*, vol. 13, no. 3, pp. 254–270, Jun 2010. [Online]. Available: <https://doi.org/10.1007/s10791-009-9112-1>
- [35] Y. Freund, R. Iyer, R. E. Schapire, and Y. Singer, "An efficient boosting algorithm for combining preferences," *Journal of machine learning research*, vol. 4, no. Nov, pp. 933–969, 2003.
- [36] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li, "Learning to rank: From pairwise approach to listwise approach," in *Proceedings of the 24th International Conference on Machine Learning*, ser. ICML '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 129–136. [Online]. Available: <https://doi.org/10.1145/1273496.1273513>
- [37] D. Metzler and W. Bruce Croft, "Linear feature-based models for information retrieval," *Information Retrieval*, vol. 10, no. 3, pp. 257–274, Jun 2007. [Online]. Available: <https://doi.org/10.1007/s10791-006-9019-z>
- [38] "Ranklib," <https://sourceforge.net/p/lemur/wiki/RankLib/>, 2020.
- [39] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [40] P. Nemenyi, *Distribution-free Multiple Comparisons*. Princeton University, 1963.
- [41] J. Torres-Jimenez and I. Izquierdo-Marquez, "Survey of covering arrays," in *2013 15th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, 2013, pp. 20–27.
- [42] M. Beller, G. Gousios, and A. Zaidman, "Oops, my tests broke the build: An explorative analysis of travis ci with github," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 356–367.
- [43] R. Somasundaram and R. Nedunchezian, "Evaluation of three simple imputation methods for enhancing preprocessing of data with missing values," *International Journal of Computer Applications*, vol. 21, 05 2011.
- [44] D. Spadini, M. Aniche, and A. Bacchelli, *PyDriller: Python Framework for Mining Software Repositories*, 2018.
- [45] J.-M. Kim and A. Porter, "A history-based test prioritization technique for regression testing in resource constrained environments," in *Proceedings of the 24th international conference on software engineering*, 2002, pp. 119–129.
- [46] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Transactions on software engineering*, vol. 27, no. 10, pp. 929–948, 2001.
- [47] J. A. Jones and M. J. Harrold, "Test-suite reduction and prioritization for modified condition/decision coverage," *IEEE Transactions on software Engineering*, vol. 29, no. 3, pp. 195–209, 2003.
- [48] D. Jeffrey and N. Gupta, "Test case prioritization using relevant slices," in *30th Annual International Computer Software and Applications Conference (COMPSAC'06)*, vol. 1. IEEE, 2006, pp. 411–420.
- [49] M. Machalica, A. Samykin, M. Porth, and S. Chandra, "Predictive test selection," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 91–100.
- [50] J. A. P. Lima and S. R. Vergilio, "Multi-armed bandit test case prioritization in continuous integration environments: A trade-off analysis," in *Proceedings of the 5th Brazilian Symposium on Systematic and Automated Software Testing*, 2020, pp. 21–30.
- [51] L. Rosenbauer, A. Stein, R. Maier, D. Pätz, and J. Hähner, "Xcs as a reinforcement learning approach to automatic test case prioritization," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*, 2020, pp. 1798–1806.
- [52] T. Shi, L. Xiao, and K. Wu, "Reinforcement learning based test case prioritization for enhancing the security of software," in *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, 2020, pp. 663–672.
- [53] R. Almaghairbe and M. Roper, "Separating passing and failing test executions by clustering anomalies," *Software Quality Journal*, vol. 25, no. 3, pp. 803–840, 2017.
- [54] R. Carlson, H. Do, and A. Denton, "A clustering approach to improving test case prioritization: An industrial case study," in *ICSM*, vol. 11, 2011, pp. 382–391.
- [55] S. Chen, Z. Chen, Z. Zhao, B. Xu, and Y. Feng, "Using semi-supervised clustering to improve regression test selection techniques," in *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 2011, pp. 1–10.
- [56] P. Kandil, S. Moussa, and N. Badr, "Cluster-based test cases prioritization and selection technique for agile regression testing," *Journal of Software: Evolution and Process*, vol. 29, no. 6, p. e1794, 2017.
- [57] Z. Khalid and U. Qamar, "Weight and cluster based test case prioritization technique," in *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*. IEEE, 2019, pp. 1013–1022.
- [58] Y. Wang, Z. Chen, Y. Feng, B. Luo, and Y. Yang, "Using weighted attributes to improve cluster test selection," in *2012 IEEE Sixth International Conference on Software Security and Reliability*. IEEE, 2012, pp. 138–146.
- [59] S. Yoo, M. Harman, P. Tonella, and A. Susi, "Clustering test cases to achieve effective and scalable prioritisation incorporating expert knowledge," in *Proceedings of the eighteenth international symposium on Software testing and analysis*, 2009, pp. 201–212.
- [60] M. Hasnain, M. F. Pasha, C. H. Lim, and I. Ghan, "Recurrent neural network for web services performance forecasting, ranking

- and regression testing," in *2019 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. IEEE, 2019, pp. 96–105.
- [61] H. Jahan, Z. Feng, S. Mahmud, and P. Dong, "Version specific test case prioritization approach based on artificial neural network," *Journal of Intelligent & Fuzzy Systems*, vol. 36, no. 6, pp. 6181–6194, 2019.
 - [62] R. Lachmann, S. Schulze, M. Nieke, C. Seidl, and I. Schaefer, "System-level test case prioritization using machine learning," in *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2016, pp. 361–368.
 - [63] M. Mahdiah, S.-H. Mirian-Hosseiniabadi, K. Etemadi, A. Nosrati, and S. Jalali, "Incorporating fault-proneness estimations into coverage-based test case prioritization methods," *Information and Software Technology*, vol. 121, p. 106269, 2020.
 - [64] S. Mirarab and L. Tahvildari, "An empirical study on bayesian network-based approach for test case prioritization," in *2008 1st International Conference on Software Testing, Verification, and Validation*. IEEE, 2008, pp. 278–287.
 - [65] T. B. Noor and H. Hemmati, "Studying test case failure prediction for test case prioritization," in *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2017, pp. 2–11.
 - [66] F. Palma, T. Abdou, A. Bener, J. Maidens, and S. Liu, "An improvement to test case failure prediction in the context of test case prioritization," in *Proceedings of the 14th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2018, pp. 80–89.
 - [67] M. M. Sharma and A. Agrawal, "Test case design and test case prioritization using machine learning," *International Journal of Engineering and Advanced Technology*, vol. 9, no. 1, pp. 2742–2748, 2019.
 - [68] A. Singh, R. K. Bhatia, and A. Singhrova, "Machine learning based test case prioritization in object oriented testing," *International Journal of Recent Technology and Engineering*, vol. 8, no. 3, pp. 700–707, 2019.
 - [69] P. Tonella, P. Avesani, and A. Susi, "Using the case-based ranking methodology for test case prioritization," in *2006 22nd IEEE International Conference on Software Maintenance*, 2006, pp. 123–133.
 - [70] H. Aman, S. Amasaki, T. Yokogawa, and M. Kawahara, "A comparative study of vectorization-based static test case prioritization methods," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2020, pp. 80–88.
 - [71] S. W. Thomas, H. Hemmati, A. E. Hassan, and D. Blostein, "Static test case prioritization using topic models," *Empirical Software Engineering*, vol. 19, no. 1, pp. 182–212, 2014.
 - [72] F. Rahman and P. Devanbu, "How, and why, process metrics are better," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 432–441.
 - [73] M. di Biase, A. Rastogi, M. Bruntink, and A. van Deursen, "The delta maintainability model: Measuring maintainability of fine-grained code changes," in *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, 2019, pp. 113–122.
 - [74] D. Di Nucci, F. Palomba, S. Siravo, G. Bavota, R. Oliveto, and A. De Lucia, "On the role of developer's scattered changes in bug prediction," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2015, pp. 241–250.

APPENDIX A

METRIC DEFINITIONS

In this section, we provide definitions of the metrics used in our test case feature model. We use the standard definitions from the relevant sources [21], [72], [44], [73], [74] and highlight and justify our definitions when they deviate from the standard ones. The metrics are defined based on a source code file. However, similar definitions can be provided based on a class or a method. In Table 15 and Table 16, we also provide summarized descriptions of the feature groups and the metrics, respectively.

A.1 Complexity Metrics

A.1.1 Program Size

- **CntDeclFunction:** Number of functions in a file.
- **LoC:** Number of all lines in a file. [aka NL]
- **LoCBlank:** Number of blank lines in a file. [aka BLoC]
- **LoCCode:** Number of lines containing source code in a file. [aka LoC]
- **LoCCodeDecl:** Number of lines containing declarative source code in a file.
- **LoCCodeExe:** Number of lines containing executable source code in a file.
- **LoCComment:** Number of lines containing comments in a file. [aka CLoC]
- **CntStmnt:** Number of statements in a file.
- **CntStmntDecl:** Number of declarative statements in a file.
- **CntStmntExe:** Number of executable statements in a file.
- **RatioCommentToCode:** Ratio of comment lines to code lines in a file.

A.1.2 McCabe's Cyclomatic Complexity

- **MaxCyclomatic:** Maximum cyclomatic complexity⁵ of all nested functions or methods in a file.
- **MaxCyclomaticModified:** Maximum modified cyclomatic complexity of nested functions or methods in a file.
- **MaxCyclomaticStrict:** Maximum strict cyclomatic complexity of nested functions or methods in a file.
- **MaxEssential:** Maximum essential complexity of all nested functions or methods in a file.
- **MaxNesting:** Maximum nesting level of control constructs in a file.
- **SumCyclomatic:** Sum of cyclomatic complexity of all nested functions or methods in a file. [aka WMC]
- **SumCyclomaticModified:** Sum of modified cyclomatic complexity of all nested functions or methods in a file.
- **SumCyclomaticStrict:** Sum of strict cyclomatic complexity of all nested functions or methods in a file.
- **SumEssential:** Sum of essential complexity of all nested functions or methods in a file.

A.1.3 Object-oriented Metrics

- **CntDeclClass:** Number of classes in a file.
- **CntDeclClassMethod:** Number of class methods in a file.

- **CntDeclClassVariable:** Number of class variables in a file.
- **CntDeclExecutableUnit:** Number of program units with executable code in a file.
- **CntDeclInstanceMethod:** Number of instance methods in a file. [aka NIM]
- **CntDeclInstanceVariable:** Number of instance variables in a file. [aka NIV]
- **CntDeclMethod:** Number of local methods in a file.
- **CntDeclMethodDefault:** Number of local default methods in a file.
- **CntDeclMethodPrivate:** Number of local private methods in a file. [aka NPM]
- **CntDeclMethodProtected:** Number of local protected methods in a file.
- **CntDeclMethodPublic:** Number of local public methods in a file. [aka NPRM]

A.2 Process Metrics

- **Commit Count:** The number of commits made to a file.
- **Distinct Dev Cnt:** The cumulative number of distinct developers who contributed to this file up to this build/release.
- **Owner's Contributed Lines:** The percentage of lines authored by the highest contributor of a file (the contributor with the most authored lines). Both added and deleted lines are counted as authored lines.
- **Minor Contributor Cnt:** The number of contributors who authored less than 5% of the code in the file.
- **Owner's Experience:** Measures the experience of the highest contributor of the file using the percent of lines he/she authored in the project up to this build/release.
- **All Committer's Experience:** The geometric mean of experience of all the developers of the file.

A.3 Change Metrics

- **Lines Added and Deleted:** The added and deleted lines in the file in this build.
- **Added and Deleted Change Scattering:** Given the added and deleted changes of a source file in a build, this metric measures the scattering of changes in the file. Assuming $CH = \{ch_1, ch_2, \dots, ch_n\}$ is the set of added/deleted code chunks in file f , we define Change Scattering (CS) as follows.

$$CS(f) = \frac{|CH|}{\binom{|CH|}{2}} \times \sum_{\forall ch_i, ch_j \in CH} \text{dist}(ch_i, ch_j)$$

where $\text{dist}(ch_i, ch_j)$ computes the distance between the two code chunks ch_i and ch_j and is defined as follows.

$$\text{dist}(ch_i, ch_j) = |\text{line}(ch_i) - \text{line}(ch_j)|$$

where $\text{line}(ch_i)$ computes line number of the beginning of code chunk ch_i . If there is only one code chunk in the changed file f (i.e., $|CH| = 1$), then $CS(f) = 0$.

The multiplication factor at the beginning of CS's formula has two objectives: (i) normalizing the distances between the code chunks by the number of pairs of code chunks modified in a build (reflected in the denominator), (ii) assigning a higher scattering to files including a higher number of changed code chunks

5. <https://support.scitools.com/t/understanding-mccabe-cyclomatic-complexity/69>

Table 15: Test case feature groups.

Group(s)	Description
TES_COM, TES_PRO, TES_CHN	Features characterizing the source code of test cases that are calculated based on code metrics (complexity, process, and change metrics).
REC	Features characterizing test cases based on their execution records (time and verdict) in builds.
F_COV	Features capturing the file coverage of test cases in changed and impacted files.
COD_COV_COM, COD_COV_PRO, COD_COV_CHN	Features characterizing the source code of the changed and impacted files covered by test cases, which are calculated based on code metrics (complexity, process, and change metrics).
DET_COV	Features based on the history of faults present in source files and detected by test cases.

Table 16: Complexity, process, and change metrics of source code.

Group	Metrics	Description
Complexity (COM)	Program Size: CntDeclFunction, LoC, LoCBlank, LoCCode, LoCCodeDecl, LoCCodeExe, LoCComment, CntStmt, CntStmtDecl, CntStmtExe, RatioCommentToCode McCabe’s Cyclomatic Complexity: MaxCyclomatic, MaxCyclomaticModified, MaxCyclomaticStrict, MaxEssential, MaxNesting, SumCyclomatic, SumCyclomaticModified, SumCyclomaticStrict, SumEssential Object-oriented Metrics: CntDeclClass, CntDeclClassMethod, CntDeclClassVariable, CntDeclExecutableUnit, CntDeclInstanceMethod, CntDeclInstanceVariable, CntDeclMethod, CntDeclMethodDefault, CntDeclMethodPrivate, CntDeclMethodProtected, CntDeclMethodPublic	This metric group includes: 1) Size metrics related to the number of lines of code, declarations, statements, and files. 2) Complexity metrics related to the control flow graph of methods. 3) Metrics based on object-oriented constructs.
Process (PRO)	CommitCnt, DistinctDevCnt, OwnerContributedLines, MinorContributorCnt, OwnerExperience, AllCommittersExperience	Metrics based on the history of the development process of files in the SUT.
Change (CHN)	LinesAdded, LinesDeleted, ChangedCodeScattering, DMMUnitComplexity, DMMUnitInterfacing, DMMUnitSize	Metrics focused on changes of the last version of the system under test.

(reflected in the numerator).

This metric is inspired by previous work [74] which proposes structural scattering for computing a developer’s change scattering in a given period of time.

- **Delta Maintainability Metric (DMM):** In one sentence, DMM is the proportion of low-risk change in a commit. The resulting value ranges from 0.0 (all changes are risky) to 1.0 (all changes have a low risk). DMM was originally calculated at the method level, and to calculate it at the commit level, we use

$$DMM = \frac{lr_chn}{lr_chn + hr_chn}$$

where lr_chn and hr_chn stand for the number of high-risk and low-risk changes, respectively. A low-risk change is defined as adding low-risk code or removing high-risk code, whereas a high-risk change is defined as adding high-risk code or removing low-risk code.

The DMM can be used on arbitrary properties that can be determined at the method (unit) level. The PyDriller [44] OS-DMM implementation supports three properties:

- **DMMUnitComplexity:** Cyclomatic complexity of method; low risk threshold 5.
- **DMMUnitInterfacing:** The number of method’s parameters; low risk threshold 2.
- **DMMUnitSize:** Method size in lines of code; low risk threshold 15.

For instance, removing lines of code from a 20-line-

method would be a low-risk change. However, adding lines of code to a method with three parameters counts as a high-risk change.

APPENDIX B USAGE FREQUENCIES OF ALL FEATURES

As a supplement to research question 2.4, which is discussed and analyzed in Section 4.4.2 of the paper, Table 17 shows the average usage frequency for all individual features in this study across all trained RF ranking models for all builds.

Table 17: Usage frequency of individual features in models that were trained on the full feature set. The *Avg. Freq.* column shows the average usage frequency of a feature across all trained RF ranking models for all builds.

Feature Group	Feature	Avg. Freq.	Feature Group	Feature	Avg. Freq.
REC	Age	17034	COD_COV_COM	CountDeclClass	309
TES_PRO	OwnersExperience	10618	COD_COV_COM	CountLineCodeDecl	302
TES_PRO	AllCommittersExperience	7643	COD_COV_COM	CountLineBlank	293
REC	TotalMaxExeTime	5409	COD_COV_COM	CountDeclMethodPublic	292
REC	LastExeTime	4095	COD_COV_COM	CountStmtDecl	291
TES_PRO	OwnersContribution	3997	COD_COV_COM	CountDeclFunction	287
TES_PRO	CommitCount	3986	COD_COV_COM	CountLine	277
REC	TotalAvgExeTime	3977	COD_COV_COM	CountDeclMethod	273
REC	RecentAvgExeTime	3850	COD_COV_COM	CountLineCodeExe	272
REC	RecentMaxExeTime	3741	REC	RecentExcRate	270
TES_COM	RatioCommentToCode	3695	COD_COV_COM	CountDeclExecutableUnit	261
TES_COM	CountStmtDecl	3383	COV	ChnScoreSum	261
TES_COM	CountLineCodeDecl	3342	COD_COV_COM	SumEssential	255
TES_COM	CountLineBlank	3149	COD_COV_COM	CountLineCode	248
TES_COM	CountStmtExe	2825	COD_COV_COM	CountStmtExe	248
TES_COM	CountLine	2792	COD_COV_COM	CountStmt	248
TES_COM	CountLineCodeExe	2688	COD_COV_PRO	AllCommittersExperience	243
TES_COM	CountLineComment	2668	COD_COV_COM	SumCyclomaticStrict	238
TES_COM	CountStmt	2658	TES_COM	CountDeclMethodDefault	238
TES_COM	CountLineCode	2453	COD_COV_COM	SumCyclomatic	230
TES_COM	CountDeclMethodPublic	1529	REC	RecentAssertRate	230
TES_COM	CountDeclInstanceVariable	1489	COD_COV_COM	SumCyclomaticModified	228
TES_PRO	DistinctDevCount	1374	COD_COV_PRO	OwnersExperience	228
TES_COM	SumCyclomaticStrict	1315	COD_COV_CHN	LinesAdded	224
REC	LastTransitionAge	1284	COD_COV_COM	RatioCommentToCode	207
TES_COM	SumCyclomatic	1281	COD_COV_PRO	OwnersContribution	196
TES_COM	SumCyclomaticModified	1280	COD_COV_CHN	LinesDeleted	192
TES_COM	CountDeclInstanceMethod	1273	COD_COV_CHN	AddedChangeScattering	168
REC	LastFailureAge	1268	TES_COM	MaxEssential	157
REC	TotalTransitionRate	1141	COD_COV_CHN	DeletedChangeScattering	150
TES_PRO	MinorContributorCount	1134	COD_COV_COM	CountDeclClassVariable	131
TES_COM	CountDeclMethod	1068	COD_COV_COM	MaxNesting	130
REC	TotalFailRate	1058	COD_COV_COM	CountDeclInstanceVariable	128
TES_COM	SumEssential	1055	DET_COV	Faults	126
TES_COM	CountDeclExecutableUnit	1039	COD_COV_PRO	CommitCount	125
TES_COM	CountDeclFunction	1035	COD_COV_COM	MaxEssential	124
TES_COM	CountDeclClassVariable	995	COD_COV_CHN	DMMSize	119
COD_COV_PRO	OwnersExperience	971	COD_COV_COM	CountLineComment	119
COD_COV_PRO	AllCommittersExperience	908	COD_COV_COM	CountDeclClassMethod	119
TES_COM	MaxCyclomaticStrict	845	COD_COV_COM	MaxCyclomaticStrict	115
REC	TotalExcRate	843	COV	ChnCount	115
COD_COV_COM	RatioCommentToCode	797	COD_COV_COM	MaxCyclomaticModified	114
TES_COM	MaxCyclomatic	784	COD_COV_COM	MaxCyclomatic	114
TES_COM	CountDeclMethodPrivate	777	COD_COV_COM	CountDeclMethodPrivate	112
TES_COM	MaxCyclomaticModified	769	COD_COV_COM	CountDeclInstanceMethod	108
COV	ImpScoreSum	761	TES_CHN	LinesAdded	108
TES_COM	CountDeclClass	749	COD_COV_CHN	DMMComplexity	107
TES_COM	MaxNesting	740	COD_COV_PRO	DistinctDevCount	105
TES_COM	CountDeclClassMethod	661	COD_COV_PRO	MinorContributorCount	105
REC	MaxTestFileFailRate	645	COD_COV_COM	CountDeclClass	101
REC	TotalAssertRate	612	COD_COV_COM	CountDeclMethodPublic	100
REC	MaxTestFileTransitionRate	597	TES_CHN	LinesDeleted	100
COD_COV_PRO	OwnersContribution	597	COD_COV_COM	CountDeclMethodDefault	96
COV	ImpCount	507	COD_COV_CHN	DMMInterfacing	94
COD_COV_PRO	CommitCount	458	COD_COV_COM	CountDeclMethodProtected	94
COD_COV_COM	CountDeclClassVariable	455	COD_COV_COM	CountLineCodeExe	93
COD_COV_COM	CountDeclMethodDefault	455	COD_COV_COM	CountLineCodeDecl	92
COD_COV_COM	CountDeclClassMethod	451	COD_COV_COM	CountLine	90
REC	LastVerdict	443	COD_COV_COM	SumEssential	90
REC	RecentFailRate	441	COD_COV_COM	CountStmtExe	88
COD_COV_COM	CountDeclInstanceVariable	437	COD_COV_COM	CountLineBlank	88
COD_COV_COM	CountLineComment	418	COD_COV_COM	CountLineCode	87
DET_COV	Faults	403	COD_COV_COM	SumCyclomaticStrict	87
COD_COV_PRO	DistinctDevCount	389	COD_COV_COM	CountDeclFunction	87
COD_COV_PRO	MinorContributorCount	386	COD_COV_COM	CountDeclExecutableUnit	86
TES_COM	CountDeclMethodProtected	360	COD_COV_COM	CountStmtDecl	86
COD_COV_COM	CountDeclInstanceMethod	359	COD_COV_COM	CountStmt	86
REC	RecentTransitionRate	355	COD_COV_COM	SumCyclomatic	85
COD_COV_COM	MaxEssential	352	COD_COV_COM	CountDeclMethod	85
COD_COV_COM	CountDeclMethodPrivate	351	TES_CHN	AddedChangeScattering	85
COD_COV_COM	MaxCyclomaticStrict	332	COD_COV_COM	SumCyclomaticModified	84
COD_COV_COM	MaxNesting	331	TES_CHN	DeletedChangeScattering	78
COD_COV_COM	MaxCyclomatic	325	TES_CHN	DMMSize	70
COD_COV_COM	CountDeclMethodProtected	317	TES_CHN	DMMInterfacing	47
COD_COV_COM	MaxCyclomaticModified	309	TES_CHN	DMMComplexity	46